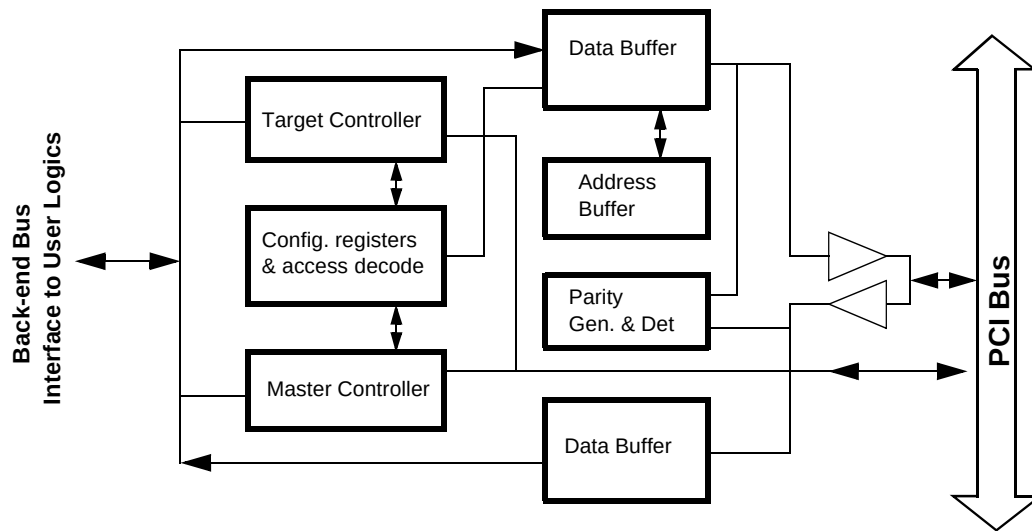




FEATURES

- Compliant with PCI specification 2.2/2.3/3.0 protocol
- Designed for ASIC and FPGA implementations in various system environments.
- Combines bus master and bus target functions in one core.
- Supports burst transfer to maximize memory bandwidth.
- Zero wait state PCI data transfer. Up to 133Mbyte/sec at 33Mhz and 266Mbyte/sec at 66Mhz.
- Supports target retry, disconnect and target abort.
- Automatic transfer restart on target retry and disconnect.
- Concurrent bus master and target function.
- Write buffer for target write data posting to increase PCI bus performance.
- Responds to standard PCI configuration access.
- Supports all PCI specific configuration registers.
- User controlled base address register sizing and mapping.
- Retry counter to limit bus access to non-responsive target device.
- PCI status directly available to user logic for interrupt generation.
- Option for integrated DMA controller.
- Option for AMBA AHB and other CPU interface.

BLOCK DIAGRAM





1. Pin Descriptions

Signal name with “_B” or “#” suffixes indicates active low signals.

Table 1: User Interface for Master Operations

Name	Type	Descriptions																																				
H_AIN[31:0]	I	User Interface Address input. Driven by the User device to the PCI master/target. It is used for transferring address of all master cycles.																																				
H_ADSH#	I	Address Strobe. It is asserted by the user logic master to request read/write transfer on the PCI bus. Address and other transfer control signals are available on the user interface at the same time interface.																																				
H_BEIN[3:0]#	I	Byte Enable. Indicates the active bytes for PCI read and write cycles.																																				
H_BLAST#	I	Burst Last. Asserted by the user interface master to terminate a master read/write operation immediately before the H_SIZE counter expires.																																				
H_CMD[1:0]	I	Command Type. This signal is used in conjunction with H_MIO and H_WRIN to define the PCI command to be issued as a master according to the following table: <table><tr><td>H_CMD</td><td>H_MIO</td><td>H_WRIN</td><td>PCI Command</td></tr><tr><td>00</td><td>1</td><td>0</td><td>memory read</td></tr><tr><td>01</td><td>1</td><td>0</td><td>memory read multiple</td></tr><tr><td>10</td><td>1</td><td>0</td><td>memory read line</td></tr><tr><td>00</td><td>1</td><td>1</td><td>memory write</td></tr><tr><td>10</td><td>1</td><td>1</td><td>memory write Invalidate</td></tr><tr><td>00</td><td>0</td><td>0</td><td>IO read</td></tr><tr><td>01</td><td>0</td><td>0</td><td>Interrupt ack</td></tr><tr><td>00</td><td>0</td><td>1</td><td>IO write</td></tr></table> Other combinations are reserved. In most applications, H_CMD can be hardwired to “00”.	H_CMD	H_MIO	H_WRIN	PCI Command	00	1	0	memory read	01	1	0	memory read multiple	10	1	0	memory read line	00	1	1	memory write	10	1	1	memory write Invalidate	00	0	0	IO read	01	0	0	Interrupt ack	00	0	1	IO write
H_CMD	H_MIO	H_WRIN	PCI Command																																			
00	1	0	memory read																																			
01	1	0	memory read multiple																																			
10	1	0	memory read line																																			
00	1	1	memory write																																			
10	1	1	memory write Invalidate																																			
00	0	0	IO read																																			
01	0	0	Interrupt ack																																			
00	0	1	IO write																																			
H_DIN[31:0]	I	Data input. It transfers data during master write and target read operation. Data bits 7-0 is the least significant byte (byte 0) and data bits 31-24 is the most significant byte (byte 3).																																				

**Table 1: User Interface for Master Operations**

Name	Type	Descriptions
H_DONE#	O	H_DONE# is asserted when the transaction is completed in the PCI bus. During write operation, H_RDYM# is asserted as soon as the last data is written into the write buffer and H_DONE# is asserted when the transfer of the last data is completed on the PCI bus.
H_DOUT[31:0]	O	Data output. Driven by the PCI master/target to the user logic. It is used for transferring data during target write and for transferring data during master read.
H_MIO	I	Memory and IO Select. This signal selects between the memory (high) and IO(low) address space.
H_RDYM#	O	This signals indicates that data on the H_DIN bus has been accepted for write or H_DOUT is completed for read by the master/target.
H_SIZE[15:0]	I	H_SIZE indicates the length of the burst transfer, measured in 32-bit words. If it has a value of 1, it is a single transfer on the user interface. If it has other values, H_SIZE is the length of the transfer. If H_SIZE is all zero, the transfer size is not limited. In this case, H_BLAST# is used to terminate the transfer when determined by the user logic.
H_WRIN	I	This signal is sample with H_ADSh# for PCI access. It is generated by the user interface controller to indicate if the transfer requested to the PCI master/target. It is low for read and high for write.

Table 2: User Interface for Target Operations

Name	Type	Descriptions
H_ABORT#	I	Asserted by the user logic to signal a PCI target abort during target access.
H_AOUT[31:0]	O	Address output bus. Driven by the PCI target to the user logic. It is used for transferring address of target read and target write.



Table 2: User Interface for Target Operations

Name	Type	Descriptions
H_ADSM#	O	It is asserted by the PCI master/target to indicate a target read/write access.
H_BEOUT[3:0]#	O	Byte Enable for target access.
H_BII#	I	Burst Inhibit Input. Asserted by the user interface slave to indicate that it does not support burst transfer for the current target access.
H_BOFF#	O	Back Off. If this signal is asserted by the master/target at the same time as H_DONE# is asserted, it indicates that an abnormal termination has occurred at the PCI bus and the current transfer is aborted. If it is asserted at any time when H_DONE# is not asserted, it indicates that an incoming PCI request is pending or the master/target is busy.
H_CS#[1:0]	O	Target chip select. Asserted by the master/target to indicate that the user interface is being accessed by another PCI master as a target of the transfer. The H_CS#[0] is asserted for BAR0 hit and H_CS#[1] is asserted for BAR1 hit. If more BARs are implemented, there will be multiple bits of H_CS#, one for each BAR.
H_DIN[31:0]	I	Data input. It transfers data during master write and target read operation. Data bits 7-0 is the least significant byte (byte 0) and data bits 31-24 is the most significant byte (byte 3).
H_DOUT[31:0]	O	Data output. Driven by the PCI master/target to the user logic. It is used for transferring data during target write and for transferring data during master read.
H_INTA#	I	Interrupt request from the user interface.
H_RDYH#	I	This signals indicates that data on the H_DOUT bus has been accepted for write or H_DIN is completed for read.

**Table 2: User Interface for Target Operations**

Name	Type	Descriptions
H_RETRY#	I	Retry. Asserted by the user logic to initiate a retry operation for the current target transaction. This signal could be used to postpone access on the user interface or to support delay transaction.
H_WROUT	O	This signal is valid for target access cycle. It is generated by the master/target to indicate if the target access is read (low) or write(high).

Table 3: PCI Bus Interface Signals

Name	Type	Descriptions
AD[31:0]	I/O	Address and Data are multiplexed on the same PCI pins. A bus transaction consists of an address phase followed by one or more data phases. The PCI master/target supports both read and write bursts.
C/BE[3:0]#	I/O	Bus Command and Byte Enables. They are multiplexed on the same PCI pins. During the address phase of a transaction, C/BE# define the bus command. During the data phase, C/BE# define byte enables.
CLK	I	System clock
DEVSEL#	I/O	Device Select. When actively driven, indicates the driving device has decoded its address as the target of the current access. As an input, DEVSEL# indicates whether any device on the bus has been selected.
FRAME#	I/O	Cycle Frame. It is driven by the master/target to indicate the beginning and duration of an access. FRAME# is asserted to indicate a bus transaction is beginning. While FRAME# is asserted, data transfers continue. When FRAME# is de-asserted, the transaction is in the final data phase.
GNT#	I	Grant. It indicates to the master/target that access to the PCI bus has been granted.

**Table 3: PCI Bus Interface Signals**

Name	Type	Descriptions
IDSEL	I	Initialization Device Select. It is used as a chip select during configuration read and write transactions. This signal is connected externally to one of the AD signals according to PCI device number.
INTA#	O	PCI interrupt signal. Controlled by user interface interrupt request.
IRDY#	I/O	Initiator Ready. It indicates the master/target's ability to complete the current data phase of the transaction. IRDY# is used in conjunction with TRDY# to indicate a completion of one data transfer.
PAR	I/O	Parity. It is an even parity across AD[31:0] and C/BE[3:0]# signals. It is driven by the master/target for address and write data phases, and is driven by the target for all read data phases.
PERR#	I/O	Parity Error. It is driven by the device on the PCI bus that receives data to report an error condition.
REQ#	O	PCI Bus Request. Driven by the master/target to indicate to the arbiter that it needs to use the bus.
RESET#	I	System Reset.
SERR#	O	System Error. It is driven by the device on the PCI bus that receives address parity errors, data parity errors on the special cycle command, or any other system error.
STOP#	I/O	Stop indicates the current target is requesting the master/target to stop the current transaction due to unusual conditions.
TRDY#	I/O	Target Ready. Indicates the target's ability to complete the current data phase of the transaction. It is used in conjunction with IRDY#.



Table 4: Other Signals

Name	Type	Descriptions
BAR0_SETUP[34:16]	I	Setup for BAR0. This is a static input allows the user to setup BAR0. Bit Description 34 1=prefetchable, 0=not prefetchable. Used only when BAR is memory mapped. 33 1=enable BAR0, 0=disable 32 1=map to PCI memory, 0=map to PCI IO 31:16 BAR size, “0111111111111111” = 2Gbytes “0011111111111111” = 1Gbytes “0001111111111111” = 512Mbytes “0000111111111111” = 256Mbytes “0000011111111111” = 128Mbytes “0000001111111111” = 64Mbytes “0000000111111111” = 32Mbytes “0000000011111111” = 16Mbytes “0000000001111111” = 8Mbytes “0000000000111111” = 4Mbytes “0000000000011111” = 2Mbytes “0000000000001111” = 1Mbytes “0000000000000111” = 512Kbytes “0000000000000011” = 256Kbytes “0000000000000001” = 128Kbytes “0000000000000000” = 64Kbytes others = Not allowed
BAR1_SETUP[34:16]	I	Setup for BAR1. This is a static input allows the user to setup BAR1. Bit Description 34 1=prefetchable, 0=not prefetchable. Used only when BAR is memory mapped. 33 1=enable BAR1, 0=disable 32 1=map to PCI memory, 0=map to PCI IO 31:16 BAR size, same definition as in BAR0_SETUP.
CLASS_CODE[23:0]	I	Static input specifying the class_code field of the configuration register.
DEVICE_ID[15:0]	I	Static input specifying the device_id field of the configuration register.



Table 4: Other Signals

Name	Type	Descriptions
H_CACHE_SIZE[7:0]	O	Cacheline size. This signal reflects the value of the cacheline size register in the PCI configuration register. User logic can use this information if needed.
INTERRUPT_PIN[7:0]	I	Static input specifying the interrupt_pin field of the configuration register.
REVISION_ID[7:0]	I	Static input specifying the revision_id field of the configuration register.
RUN_66MHZ	I	Static input specifying the 66Mhz field of the configuration register.
SEL_23	I	Static input select between PCI 2.2 or 2.3 version. This signal is high for version 2.3 and low for version 2.2 compliant. PCI spec 3.0 is identical to 2.3 as far as the controller core is concerned.
STATUS15, STATUS14, STATUS13, STATUS12, STATUS11, STATUS8, STATUS_RETRY	O	Output signals reflect the values of the PCI status register bit 8, and 11 - 15. STATUS_RETRY reflects the value of the retry overflow bit in the retry control register.
SUBSYSTEM_ID[15:0]	I	Static input specifying the subsystem_id field of the configuration register.
SUBVENDOR_ID[15:0]	I	Static input specifying the subsystem vendor_id field of the configuration register.
VENDOR_ID[15:0]	I	Static input specifying the vendor_id field of the configuration register.



2. Functional Descriptions

The EC220 master/target functions as a bi-directional PCI bus interface that includes the function of a PCI bus master and PCI bus target. Access can be initiated by the user logic (CPU or DMA) to access PCI memory and I/O. Master devices such as DMA controller can also reside on the user interface to initiate data transfer to and from the PCI bus. In the opposite direction, the EC220 also functions as a PCI target so other PCI bus master can read or write data to slave devices on the user interface. Zero wait state burst transfer is supported in both directions.

Different options of the PCI master/target are available for different applications. Please contact Eureka Technology for more detail information of each option.

Options
Asynchronous user interface
Direct interface to AMBA AHB, MIPS or PowerPC.
Integrated DMA controller
64-bit PCI or user interface width
Dual Address Cycle
MiniPCI support
Power Management
Cardbus and MiniPCI support
Hot Swap for Compact PCI
Host bridge functions

The following diagram indicates the bi-directional nature of the PCI master/target.

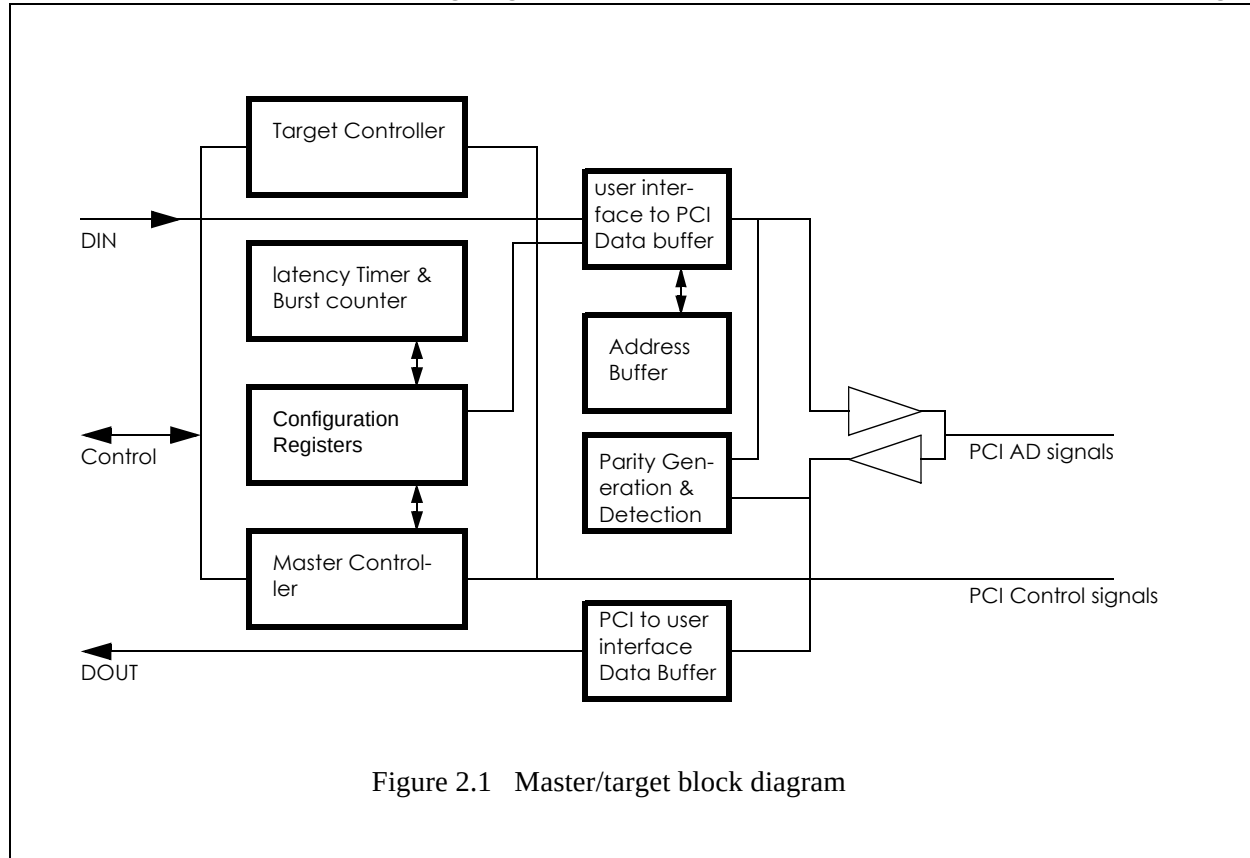


Figure 2.1 Master/target block diagram



3. Master Functions

3.1 PCI master burst read

The user logic initiates PCI master read request by asserting the H_ADSH# signal. Valid address is driven on H_AIN and control signals such as H_SIZE, H_BEIN#, H_WRIN, H_CMD and H_MIO are also driven by the user logic at the same time. As soon as the master/target samples H_ADSH# being active at the rising clock edge, it starts the bus access sequence on the PCI bus. If the PCI bus has already been granted to the master/target and the PCI bus is idle, it asserts FRAME# in within two cycles to start the transfer. If the PCI bus is not yet granted, it asserts REQ# on the PCI bus to start the bus arbitration process.

When read data returns from the PCI bus, it is driven on the H_DOUT bus and H_RDYM# is asserted to indicate valid data. If the host request is for multiple accesses, the number of PCI WORD indicated by H_SIZE will be read from the PCI bus and H_RDYM# is asserted for each transfer. For normal PCI read transfers, H_DONE# is asserted at the same time as the last H_RDYM# assertion.

If H_BOFF# is asserted at the same time with H_DONE#, it indicates that the transfer is aborted by the PCI target being accessed or if no target responds to the transfer (master abort). It is an error condition and the transfer is terminated.

The following diagram shows a 4-data burst read access initiated by the user



logic.

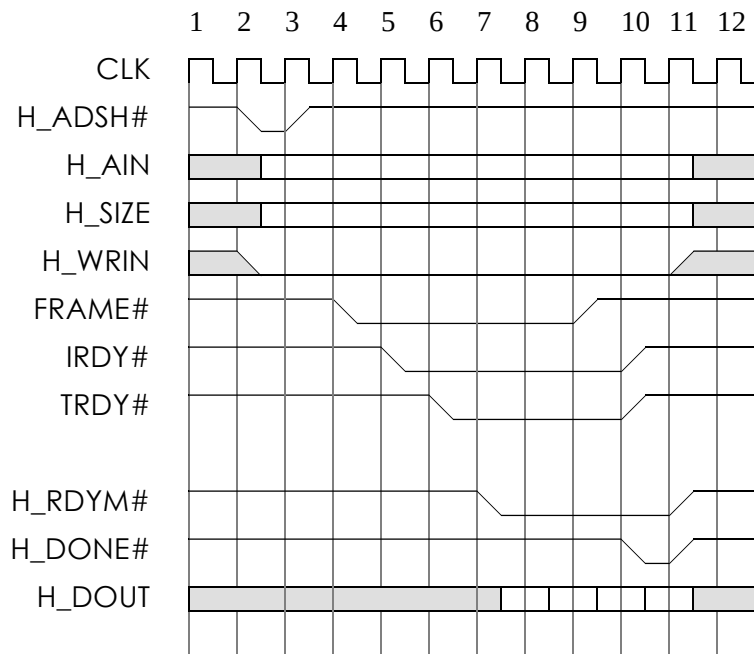


Figure 3.1 Burst read access initiated by the master/target

3.2 PCI write initiated by master/target

Write access is initiated by H_ADSH# similar to read access. Valid address is driven on H_AIN and control signals such as H_SIZE#, H_BEIN#, H_WRIN, H_CMD and H_MIO is also sampled at the same time. For all write cycles, data must be presented at the H_DIN bus one cycle after H_ADSH# is asserted. As soon as the master/target samples H_ADSH# being active at the rising clock edge, it starts the bus access sequence on the PCI bus. If the PCI bus has already been granted to the master/target, it asserts FRAME# within two cycles to start the transfer. If the PCI bus is not yet granted, it asserts REQ# on the PCI bus to start the bus arbitration process.

Data must remain valid on the user interface until H_RDYM# is asserted by the master/target. If a burst write is requested, the next write data must be driven on H_DIN after H_RDYM# is asserted.

User interface data transfer is completed when the last H_RDYM# is asserted. However, data transfer on the PCI bus is not completed until a few cycles later.



When data transfer is completed on the PCI bus, H_DONE# is asserted. The user logic cannot start another bus request until H_DONE# is asserted.

If the transfer is aborted by the target or if no target responds to the transfer (master abort), H_DONE# is asserted immediately with H_BOFF#. Target or master abort indicates system error on the PCI bus.

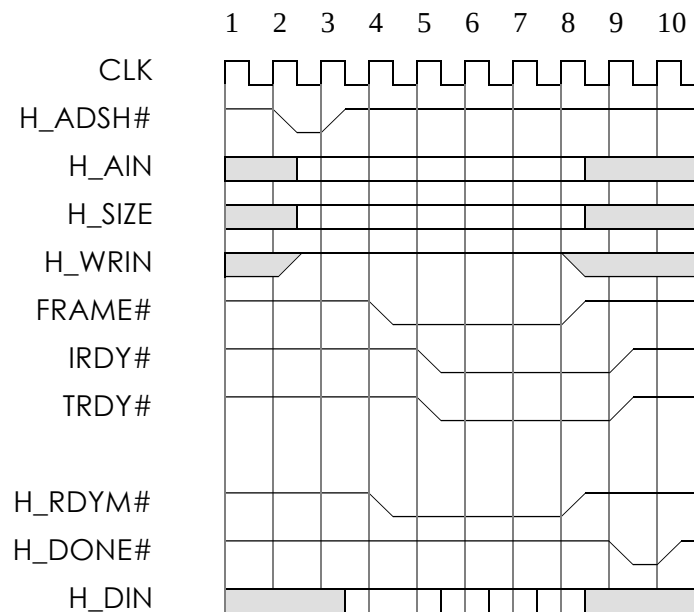


Figure 3.2 Burst write access initiated by PCI master/target

3.3 PCI write with wait states

The following diagram shows a PCI write access with wait states generated by the PCI target. To improve data bandwidth between the user interface and the PCI bus, the PCI master/target contains two write buffers. These two buffers store data to be written to the PCI target.

During the initial stages of the write transfer, the master/target asserts H_RDYM# to write into the write buffers. When data is written into the PCI bus they are removed from the write buffer. The rate of the data removed from the write buffer depends on the speed of the PCI target being written. When the write buffer becomes available, the PCI master/target asserts H_RDYM# again to write



another data into the write buffer. The double write buffer design requires much less gate count than a traditional FIFO design. It has the same advantage as a FIFO design by allowing data to enter and exit the PCI master/target simultaneously and at the same time sustain high speed data transfer such as zero wait state burst.

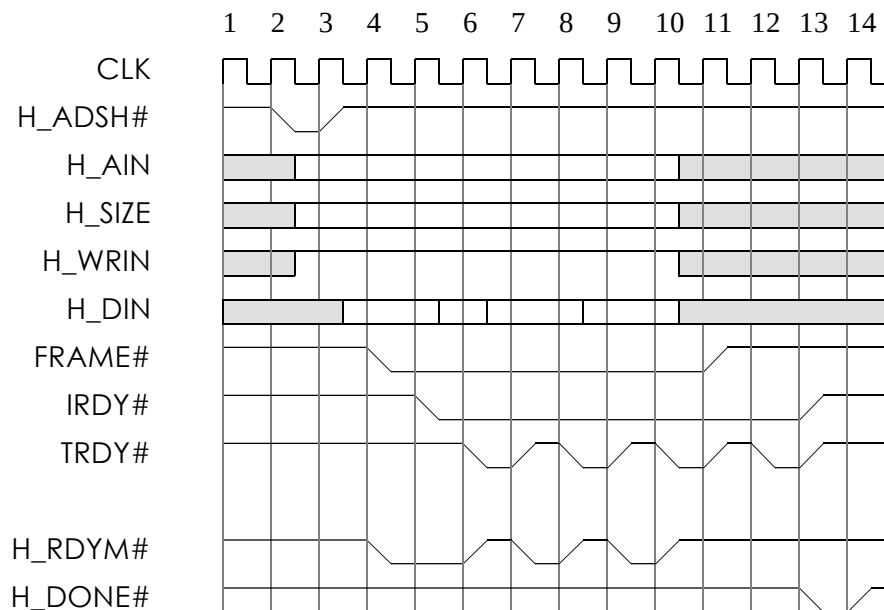


Figure 3.3 Burst write access with wait state

3.4 Auto restart

Data transfer on the PCI bus can be interrupted in many ways. For example, a target may initiate a disconnect during a burst transfer for any reason. A master/target may also terminate a transfer due to time out before the burst sequence is completed.

If any of these condition occurs before the intended number of data transfer has completed, the master/target will terminate the PCI cycle according the PCI specification and will automatically re-initiate the transfer request on the PCI bus. This function of automatic restarting removes the burden of restarting a request from the user logic and insulates it from the complexity of the PCI bus protocol.



Auto-restart is transparent to the user logic except that the user logic sees several cycles of wait states during a restart. Both read and write accesses can be re-started if they are interrupted on the PCI bus. The following diagram shows target disconnect during read and restarted by the master/target.

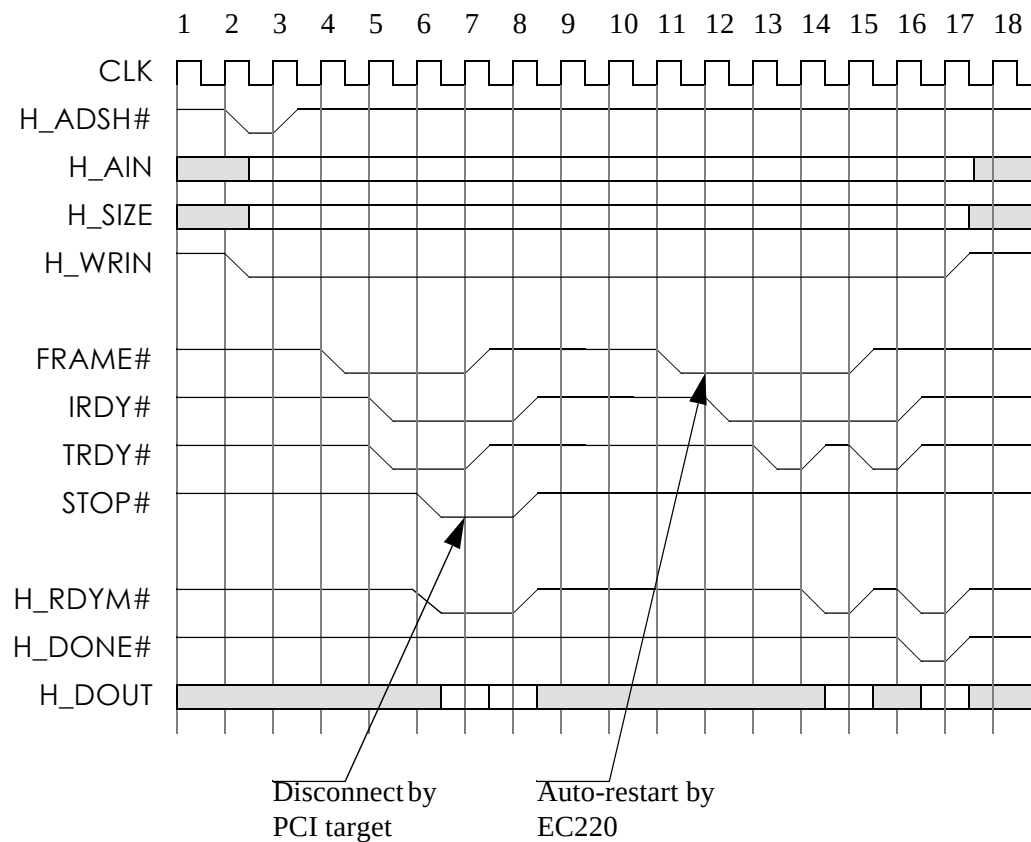


Figure 3.4 Master burst read with auto restart

3.5 Early termination of master access

The master/target supports two methods to control the burst length of any PCI master access. If the transfer size is known when the transfer starts, valid value can be driven to the H_SIZE input when H_ADSH# is asserted. The master/target contains internal counter to count the transfer size and terminates the PCI transaction when the counter expires.

The second method is drive all "0" to H_SIZE when H_ADSH# is asserted. This



way the internal counter is disabled and the PCI burst transaction runs continuously until H_BLAST# is asserted. When H_BLAST# is asserted, the master/target terminates the PCI transaction as quickly as the PCI bus protocol allows and then asserts H_DONE# when the PCI bus returns to idle.

Once H_BLAST# is asserted, it must remain asserted until the next H_RDYM# assertion. When H_BLAST# and H_RDYM# are asserted together, the transfer terminates at the user interface.

H_BLAST# can be asserted at the same time or one cycle after H_ADSh# is asserted to indicate single data transfer. However, H_BLAST# cannot change state between the second cycle after H_ADSh# and the first cycle of H_RDYM#.

H_BLAST# can be asserted by the user logic even if the H_SIZE counter is enabled. In this case, the master/target would terminate the transaction as early as the PCI bus protocol allows without waiting for the internal counter to expire. The following diagrams indicate the timing of the H_BLAST# signal.

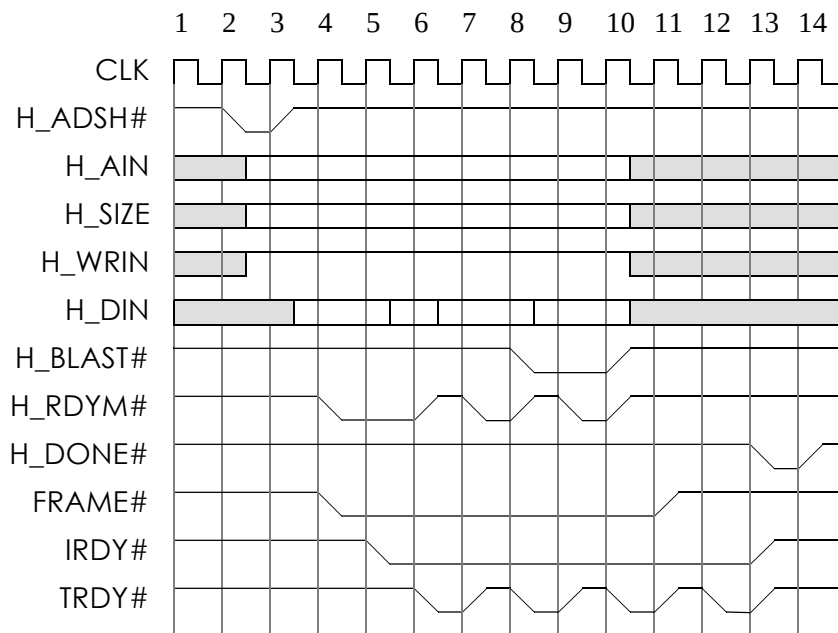


Figure 3.5 Burst write terminated with H_BLAST#



The following timing diagram shows the timing of burst read terminated by H_BLAST#. It should be note that even tough the burst read at the user interface is terminated after the second data, the read operation continues on the PCI bus until the master can de-assert FRAME# and terminates the burst read. Extra data read by the master from the PCI bus is discarded.

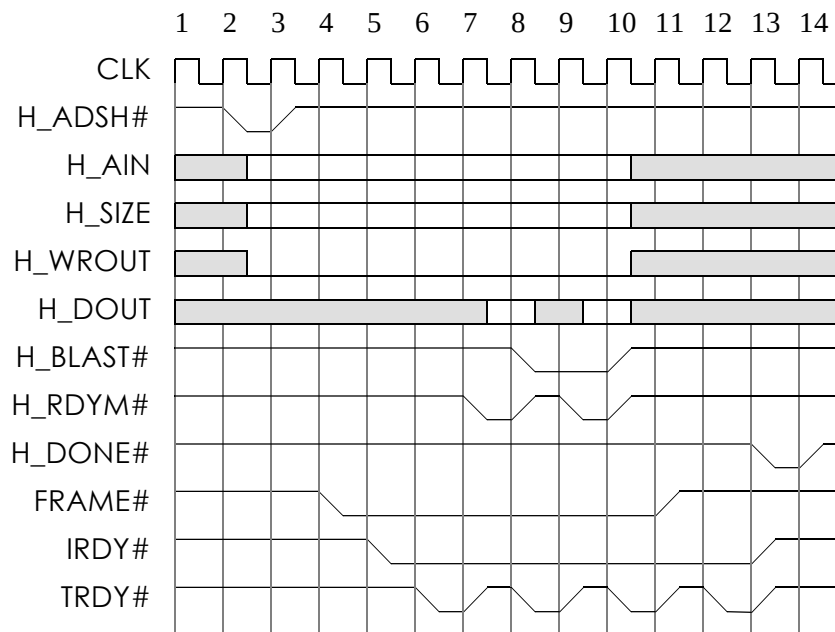


Figure 3.6 Burst read terminated with H_BLAST#

3.6 Target abort

There are several PCI mechanism for the target device to terminate data transaction other than normal termination. They are target retry, disconnect and abort. When the target signals retry or disconnect, the master/target automatically restart the transaction as explained in previous section.

If the target signals a target abort, or if no target response to the request after 6 PCI clock cycles, the master/target immediately terminates the transaction and asserts H_BOFF# and H_DONE#. When the user logic sees H_BOFF# is asserted at the same time as H_DONE#, it means that the transaction is aborted. Data received or transmitted before the target abort may not be correct. In general, a PCI target signals target abort only if the transaction is illegal and the user logic



should not attempt the transaction again.

3.7 Byte enable

Bytes within the 32-bit data width of the PCI bus can be read or written individually. When the user logic request single data transfer, the H_BEIN# signals are used to specify the byte(s) to be written or read from.

When a burst read is requested, i.e. H_SIZE not equal one, the byte enable input, H_BEIN#, is ignored and the master/target assumes that all bytes are enabled for read.

When burst write is requested, H_BEIN# is sampled at the same time as data. This way byte enable can be changed from data to data and the byte enable is propagated to the PCI bus CBE# signal with each data phase.

3.8 Burst Sequence

Burst sequence is the address order in which successive data is accessed. The PCI master/target supports linear increment burst order.

When memory address is accessed by the master/target, the two least significant bits of the PCI address are set to zero by the EC220 to indicate linear increment burst order. The two least significant bits are not forced to zero when I/O space is accessed.

The EC220 can be modified to support other burst sequence such as cache line wrap mode. Please contact Eureka Technology for more detail.



4. Target Functions

4.1 Target device read

The EC220 PCI master/target also functions as a PCI target device. It is mapped to PCI memory or I/O space through the regular base address register mechanism. PCI request to the target is forwarded to the user logic. This feature allows other PCI devices on the bus to access system memory and other system resources that resides on the user interface.

The following diagram illustrate a burst target read request from the PCI bus to the user logic. When a PCI requested to the user logic is detected, the H_BOFF# signal is first asserted to indicate the pending request. Since H_DONE# is not asserted at the same time, H_BOFF# does not indicate an error condition. Assertion of H_BOFF# without H_DONE# is a mechanism for the PCI master/target to gain control of the user interface. The H_ADSM# signal is then asserted one or more cycles later to indicate that valid address is available on the H_AOUT pins and H_BEOUT# and H_WROUT signals are valid. H_CS# is asserted at the same time as H_ADSM# and remains asserted until the end of the transfer.

For a read request, H_WROUT is low and the user logic drives the read data on to the H_DIN bus when it becomes valid and asserts H_RDYH# at the same time. In this timing diagram, both zero wait state and one wait state are used by the user logic at different data phase. More wait states can be inserted by the user logic if necessary to regulate data transfer rate.

Both burst and single data transfers are allowed. The transaction completes when



H_CS# is de-asserted.

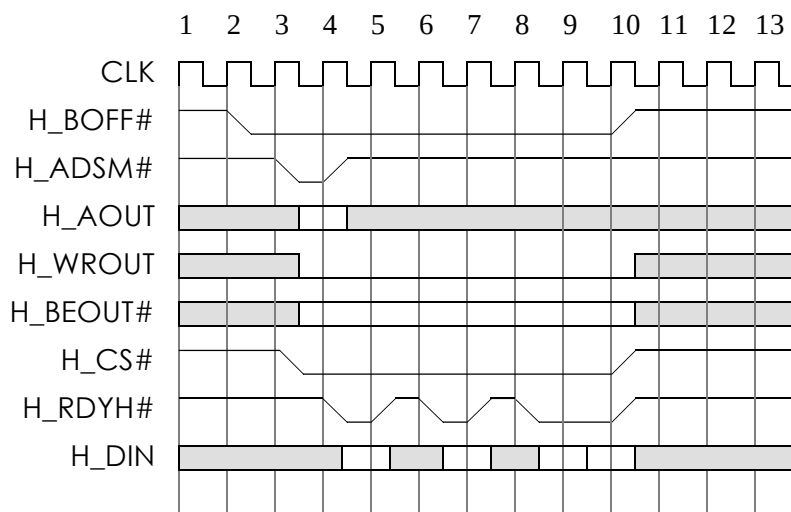


Figure 4.1 Target burst read with zero wait state

4.2 Target write

Target write is very similar to target read. It is preceded by the H_BOFF# signal. Both H_ADSM# and H_CS# are asserted to start the access. H_CS# remains asserted until the end of the transfer but H_ADSM# is asserted only for the first cycle to signal the address phase.

The following diagram shows a two data target burst write access. The cycle delay from H_BOFF# asserted to H_ADSM# asserted varies according to the exact timing sequence on the PCI bus.

At the same cycle when H_ADSM# is asserted, the write address is driven on the H_AOUT bus. Immediately in the following cycle, the write data is driven on the H_DOUT bus. The data remain valid until H_RDYH# is asserted by the user interface to indicate completion of write. If the access is a single write, the transaction terminates at the first H_RDYH#. For a burst write access, the second write data is driven on to H_DOUT in the cycle after H_RDYH# is asserted. In this example, the user interface asserts H_RDYH# in two consecutive cycle and the burst write is completed on the second H_RDYH# assertion. Wait state can be asserted by the user logic by delaying the assertion of H_RDYH#.

One should note that burst target access is supported for both read and write trans-



actions with zero or more wait states. Up to 133Mbyte/sec of data rate can be achieved at 33Mhz and 266Mbyte/sec at 66Mhz.

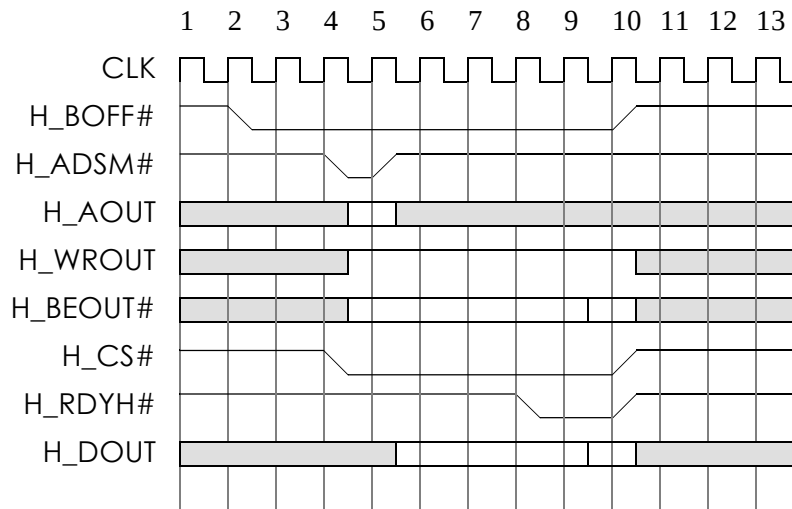


Figure 4.2 Burst target write with initial wait state

4.3 Target retry/Delay transaction on read

The PCI specification requires that each target to complete the first data transfer within 16 clock cycles. In the event that the user logic requires more time to complete the data transfers or that it cannot process the current request for other reasons, it can use the H_RETRY# pin to terminate the transfer without data. The H_RETRY# can be asserted by the user logic any time during a read transfer. It should assert H_RETRY# as soon as it detects that it cannot provide the read data within the required time.

Both PCI retry and disconnect are signals by the PCI STOP# signal asserted by the target. Such termination is called retry if STOP# is asserted before any data has been transferred. It is called disconnect if some data has already been transferred on the PCI bus when STOP# is asserted. Both retry or disconnect may be triggered by H_RETRY# depending on the timing of the H_RETRY# signal.

On the user interface, the transaction ends when H_CS# is de-asserted. The core de-asserts H_CS# one cycle after H_RETRY# is asserted. The user may assert H_RETRY# and H_RDYH# at the same time. In that case, the read data is transferred to the PCI bus and the core also signals a disconnect on the PCI bus.



Once the H_RETRY# signal is asserted, the master/target terminates the current target transaction with retry or disconnect. Target retry or disconnect are temporary termination of the PCI transfer and the bus master is required to restart the transfer in a later time. The following diagrams shows the timing of a read retry due to H_RETRY#.

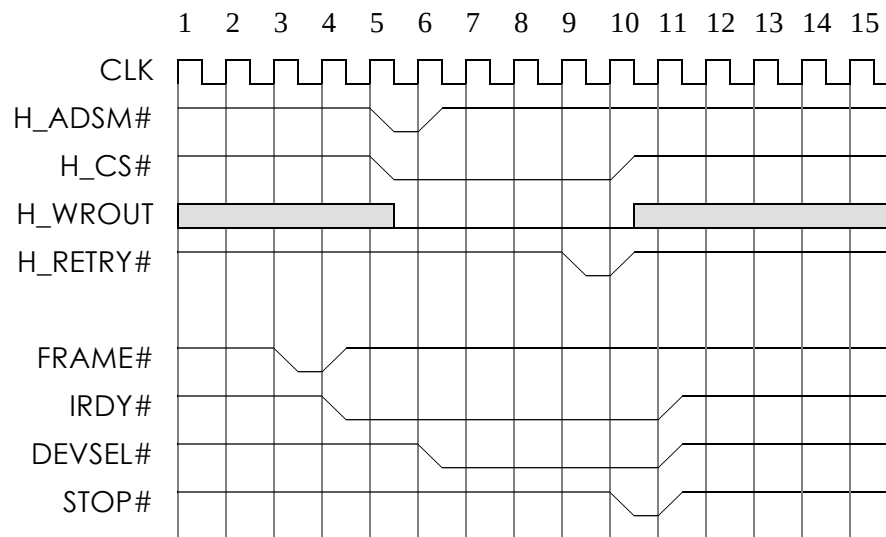


Figure 4.3 Target retry on read

4.4 Target retry on write

The user can also specifies target disconnect on write by using the H_RETRY# input. When H_RETRY# is asserted during a target write, the core terminates the current write transaction on the user interface by de-asserting H_CS#. It also signals PCI target disconnect on the PCI bus if the PCI bus transaction is still in progress.

Because the target uses posted write to process all write transfers, it may still contains data in its posted write buffer when the user interface asserts H_RETRY#. In that case the core will re-start the write access on the user interface 2 cycles after H_RETRY# is asserted to ensure that all the data in its posted write buffer is written to the user interface. Similar to retry on read, H_RETRY# and H_RDYH# may be asserted at the same time. In that event, the user interface transfers the cur-



rent write data and signals a write retry at the same time.

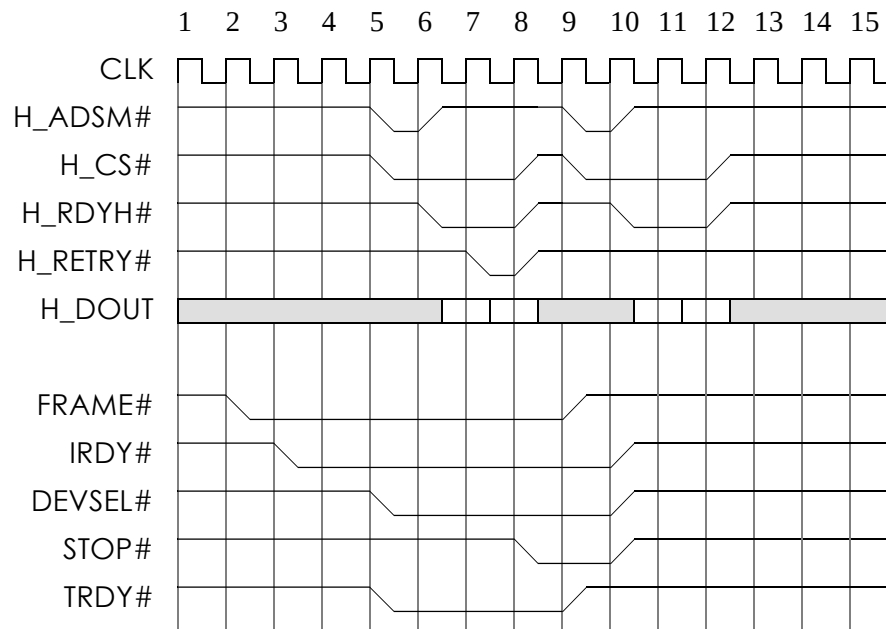


Figure 4.4 Target retry on write

4.5 H_RETRY# during bus idle

The H_RETRY# signal can also be used to by the user logic to disallow the target from accepting all PCI target request except configuration access. When the target core receives a target access, it checks the H_RETRY# signal before it is supposed to assert the H_ADSM# signal to start the transfer. If H_RETRY# is asserted, the core will not forward the transfer to the user interface. It will issue a PCI target retry without transfer any data. If it is a target write, the core will not accept any data in its posted write buffer.

4.6 Target abort

Target abort on the PCI bus signals error condition on the target so the transfer is terminated. Unlike target retry, target abort is an abnormal condition and the PCI bus master should not restart a request that has been aborted.

The user logic signals a target abort by asserting H_ABORT#. On a target read transaction, the user may assert H_ABORT# one cycle or more after H_ADSM#



is asserted. When H_ABORT# is asserted, the master/target terminates the current target transaction by issuing a PCI target abort. The timing is similar to H_RETRY# assertion except that the core signals target abort instead of retry/disconnect on the PCI bus. The read transaction terminates on the user interface at the assertion of H_ABORT#. H_ABORT# may be asserted before or after any data is transferred on the user interface.

If the transaction is a target write, the master/target will abort the transaction on the user interface. If the transaction is still in progress on the PCI bus, it will also signal target abort. There may be data written into the posted write buffer of the core when the H_ABORT# is asserted. Those data are discarded by the core when H_ABORT# is asserted. H_ABORT# may be asserted before or after any data is transferred on the user interface.

H_ABORT# should never be asserted when the user interface is idle. It can be asserted only one cycle or more after H_ADSM# is asserted and H_ABORT# and H_RDYH# may not be asserted at the same time.

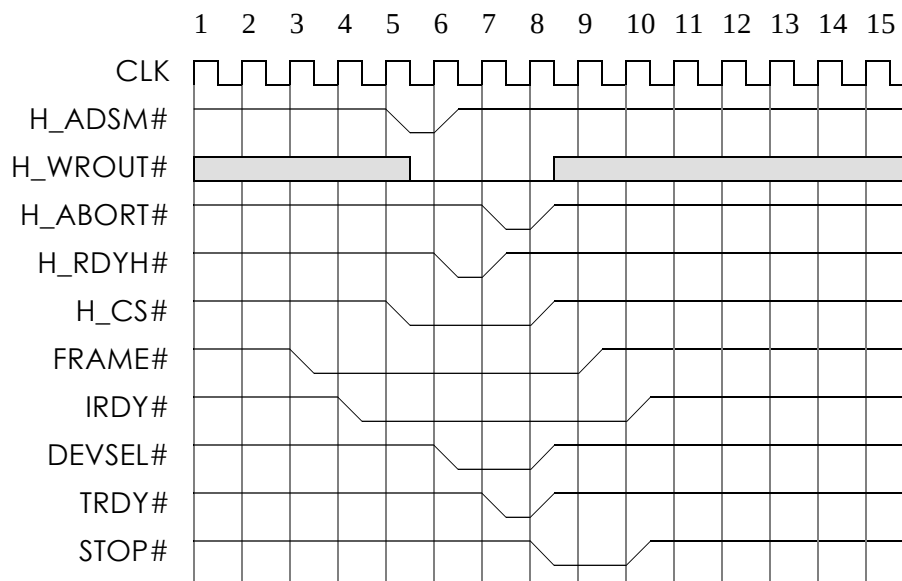


Figure 4.5 Target abort on read

4.7 Burst inhibit

The master/target is capable of handling burst data transfer both as master and as



target. The user logic, however, can determine if it allows burst data transfer on a case by case basis.

When the master/target functions as a bus master, it specifies non-burst transfer if H_SIZE contains a value of 1, otherwise the transfer is burst transfer.

When the master/target functions as a bus target, it specifies non-burst transfer if the H_BII# input is asserted. The following diagram shows a non-burst target read specified by the master/target using H_BII#.

FRAME# is asserted by the PCI master to initiate a target read from the master/target at cycle 3. The transaction is decoded by the master/target and H_ADSM# is asserted in cycle 5. At the same time the transfer address is driven on the H_AOUT bus. The user logic must decode the H_AOUT signal and determine if it support burst transfer for the current access. If burst is not allowed by the user logic, it must assert H_BII# in the following cycle. Once asserted, H_BII# must remain asserted until the end of the transfer. When the read data for the first transfer is returned from the user logic at cycle 9, H_RDYH# is asserted. The master/target asserts TRDY# on the PCI bus to signal return data and at the same time asserts STOP# to signal a target disconnect. This indicates to the bus master that it must terminate the current transfer with one read data. The master is required to restart another read access to read the remaining of the data in a later cycle.

Burst inhibit can also be used during target write to force a target write to single



data write. H_BII# timing is identical in both target read and target write.

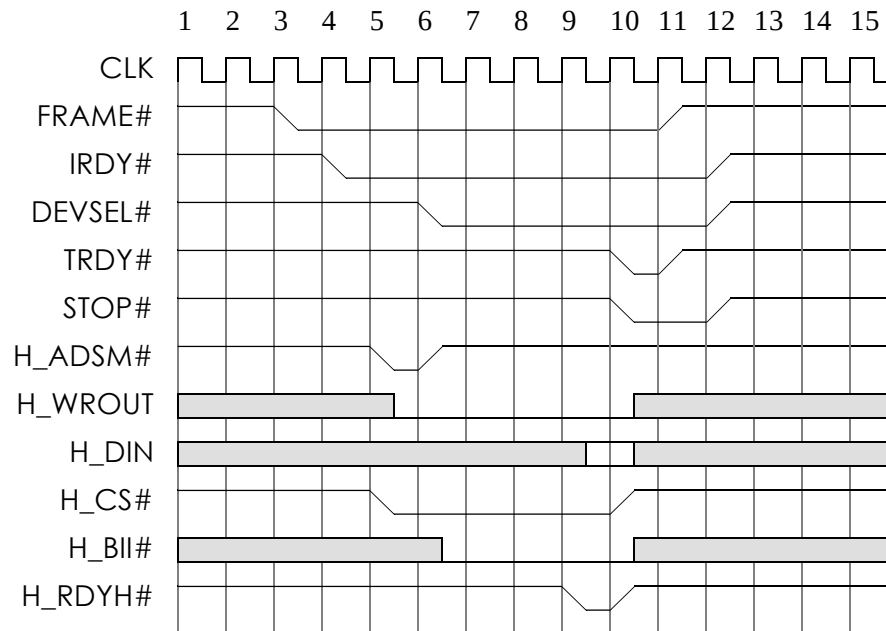


Figure 4.6 Burst inhibited by the user interface

4.8 Concurrent master/target operation

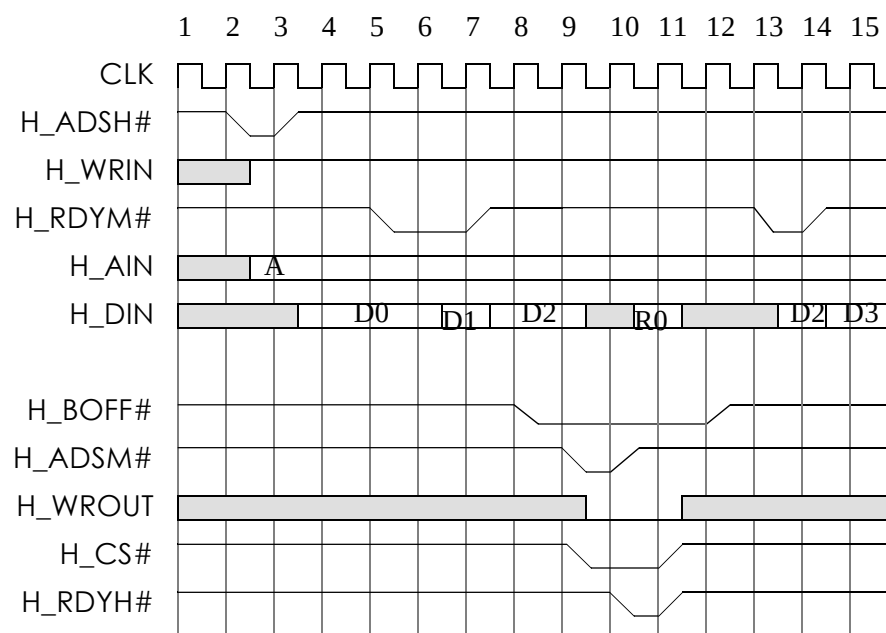
The master/target supports concurrent bus master and target operation. A user logic may request a PCI master operation to the master/target. While the master operation is in progress on the user interface, another PCI bus master may have ownership of the PCI bus and request access to the master/target. The master/target allows such concurrent function since it has dedicated resources for both the master and the target.

The following timing diagram shows one such example. The user interface asserts H_ADSH# in cycle 2 to request a master write. Write data is transferred to the master/target in cycle 5 and 6 at the assertion of H_RDYM#. Shortly afterwards the master/target may have lost ownership of the PCI bus and another bus master request reading of the master/target as a target. H_BOFF# is asserted in cycle 8 and H_ADSM# is asserted in cycle 9 to start the target read in the user interface. Read data is transferred in the user interface when H_RDYH# is asserted and the target read is completed. H_BOFF# is then de--asserted to indicate the target



access completion.

The assertion of H_BOFF# indicates to the user logic that a target access is current running in the PCI bus. H_DIN is shared by both the master write and the target read operation to provide write data and read data. When H_BOFF# is asserted, the user interface must allow the read data to be driven on the H_DIN bus. The user interface can always decline a target read by using the H_RETRY# signal



Legend for H_DIN bus:

D0 - D3: master write data

R0: target read data

Figure 4.7 Concurrent master and target operation

4.9 Interrupt

The master/target supports interrupts generated by the user logic and propagate the interrupt to the PCI bus. When the user interface request interrupts, it asserts H_INTA# and the master/target asserts INTA# at the same time. INTA# is an open drain output according to PCI specification.

In addition, the master/target contains the proper interrupt information in the PCI configuration register as required by the PCI specification. The register are in



location 3C(Hex) and 3D(Hex). Please see the configuration register section for more detail. The INTERRUPT_PIN input should be 01(hex) to indicate usage of INTA# signal.

4.10 Parity

The EC220 master/target generates the PCI parity signal, PAR, whenever it drives the PCI AD bus. If the parity enable bit in the configuration register is enabled, the master/target will also detect and signal bus parity error whenever it receives data from the PCI bus. Data parity error is signaled by asserting the PERR# signal. However, the EC220 continue to process the bus transaction even when parity error is detected.

Address parity error is also detected if the system error control bit in the configuration is enabled. The master/target signals address parity by asserting the SERR# signal. Even when address parity is detected, if the master/target is the selected device, it claims the cycle and terminate as though the address was correct.

The system handling after PERR# and SERR# detection is very dependent on overall system design. The EC220 can assists system error handling by making the PERR# and SERR# status bits of the configuration register visible to system logic according to user specification.

For details on PCI parity generation and detection, please refer to the PCI specification. The master/target, however, handles all parity generation and detection task so it is transparent to the user interface.

4.11 Transaction type

The master/target supports different transaction commands as bus master and as target. As a bus master it can initiate the following transactions:

- Memory Read
- Memory Read Line
- Memory Read Multiple
- Memory Write
- Memory Write Invalidate
- IO Read
- IO Write
- Interrupt Acknowledge

Memory-write and Memory-write-invalidate are selected by the H_CMD inputs. The Master MWI enable bit (bit 4 of command register) must be enabled for Memory-write-invalidate command to be issue. If the enable bit is zero, Memory-write command will be issued instead of Memory-write-invalidate. In order to issue memory write invalidate command, the user must specify the transfer size, H_SIZE to be multiple of the cacheline size.

Cacheline size is a register in the configuration register space. The value of this



register can be observed at the user interface through the H_CACHE_SIZE output.

Memory-read, Memory-read-line and Memory-read-multiple commands are selected by the user through the H_CMD input. The user is responsible for providing the correct transfer size, H_SIZE, for each of these commands.

As a bus target, it can receive the following transactions:

- Memory Read
- Memory Read Line
- Memory Read Multiple
- Memory Write
- Memory Write & Invalidate
- IO Read
- IO Write
- Configuration Read
- Configuration Write

Memory-read, Memory-read-line and Memory-read-multiple are processed by the target in the same way. Memory-write and Memory-write-invalidate are processed by the target in the same way. Configuration-read and Configuration-write are processed by the target locally without propagate to the user interface.

4.12 Configuration cycles

The master/target contains configuration register data as any PCI agent. It accepts configuration access initiated by other PCI devices such as the system host bridge. All configuration register access are handled locally by the master/target and are not propagated to the user interface.

The configuration mechanism implemented by the master/target is the standard type 0 configuration access.

The following diagram shows the timing of a PCI configuration read to the EC220. On the PCI bus it is very similar to a regular read except that all the read data comes internally from the master/target's own configuration space and that



the user interface is not accessed.

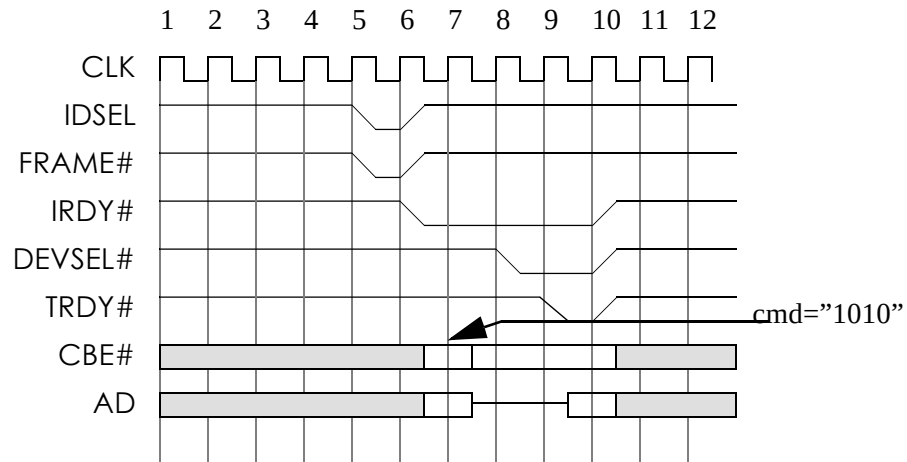


Figure 4.8 Configuration read

4.13 Configuration Registers

The EC220 master/target features all the configuration register required by the PCI specification. The following is a map of the configuration register. All configuration register accesses are processed automatically by the master/target and



does not propagate into the user interface.

Device ID		Vendor ID		00h
Status		Command		04h
Class Code			Revision ID	08h
BIST	Header	Lat. timer	Cache size	0Ch
Base Address Registers 0				10h
Base Address Registers 1				14h
Base Address Registers 2				18h
Base Address Registers 3				1Ch
Base Address Registers 4				20h
Base Address Registers 5				24h
Cardbus CIS pointer				28h
Subsystem ID		Subsystem Vendor ID		2Ch
Expansion ROM				30h
Reserved				34h
Reserved				38h
Max_Lat	Min_Gnt	Int. pin	Int. line	3Ch
Retry over-flow flag	Counter enable	Retry counter		40h

Figure 4.9 Configuration Registers



The following section describes each configuration register bits in detail:

00h: Device ID - Read-only field defined by the user using the device_id input.

Vendor ID - Read-only field defined by the user using the vendor_id input.

04h: Command Register:

Bit	Description	Default
15-11	Reserved	Read only, 0
10	This bit is used only in PCI spec 2.3. If this bit is 0, assertion of INTA#-INTD# is allowed. If this bit is 1, assertion of INTA#-INTD# is not allowed. This bit is hardwired to "0" in PCI spec 2.2	Read/write (2.3) Read only, 0 (2.2)
9	Fast back-to-back, read only unless requested by user	Read only, 0
8	SERR#	Read/write
7	Address data stepping, not needed	Read only, 0
6	Parity error response	Read/write
5	VGA palette snooping, read only unless requested by user, read only unless requested by user	Read only, 0
4	Master MWI enable.	Read/write
3	Special cycle response, read only unless requested by user	Read only, 0
2	Bus master	Read/Write
1	Memory enable	Read/Write
0	IO enable	Read/Write

06h:Status Register

Bit	Description	Default
15	Parity error	Read/Clear
14	SERR#	Read/Clear
13	Master abort	Read/Clear
12	Target abort as master	Read/Clear
11	Target abort as target	Read/Clear
10-9	DEVSEL timing	Read only, 10
8	PERR#	Read/Clear
7	Fast back-to-back, read only unless requested by user	Read only, 1



Bit	Description	Default
6	UDF, read only unless requested by user	Read only, 0
5	33/66Mhz, read only, specified by the user using the run_66mhz input.	Read only, 1/0
4	Capability list. This bit is normally 0 unless optional features such as power management or hot swap is implemented	Read only
3	This bit is used in PCI spec 2.3 only. This bit reflect the inverted value of H_INTA# input. INTA# is asserted only when this bit is "1" and interrupt disable bit command register is "0". In PCI spec 2.2 this bit is always "0".	Read only
2-0	Reserved	Read only, 0

08h: Class Code - Read-only field defined by the user using the class_code input.

Revision ID - Read-only field defined by the user using the revision_id input.

0Ch: Cache line size - All bits are read and write.

0Dh: Latency Timer - All bits are read and write.

0Ch: Header- All bits are read-only and zero unless requested by user.

0Ch: BIST - All bits are read-only and zero unless requested by user.

10h-24h: Base Address Registers. Up to 6 BAR can be defined. The standard core has two BAR built-in and the size of each BAR can be defined by the user through the BAR0_SETUP and BAR1_SETUP inputs. More than 2 BAR can be implemented. Please contact Eureka Technology for more information.

28h: Cardbus CIS Pointer - Read-only and zero

2Ch: Subsystem ID - Read-only and defined by user. using the subsystem_id input

Subsystem Vender ID - Read-only and defined by user using the subvendor_id input.

30h: Expansion ROM base address. Specified by user and implemented by Company.

34h to 38h: Reserved or not used - Read-only and zero.

3Ch: Interrupt line - read and writable.

3Dh: Interrupt pin - Read-only and defined by user.

3Eh: Minimum Grant - Read-only and specified by user.

3Fh: Maximum Latency - Read only and specified by user.

40h: Retry counter - This is a 16-bit register that specifies the maximum number



of auto-restart the PCI master will issue for each access request. If the external target issues retry that exceed this value, the master will assume that the external target has failed not retry the request again.

42h: Counter enable. Bit 0 of this register indicates if the retry counter is enabled. If this bit is “0”, the retry counter is not enabled and the PCI master does not impose any limit in the number of auto-restart due to retry. If this bit is “1”, the retry counter limit is imposed. Bit 7 to 1 of this register are not used.

43h: Retry overflow flag. Bit 0 if this register is set if the master abort a request due to retry counter limit. User can clear this bit by writing “1” to bit 0. Writing “0” has no effect. Bit 7 to 1 of this register are not used.

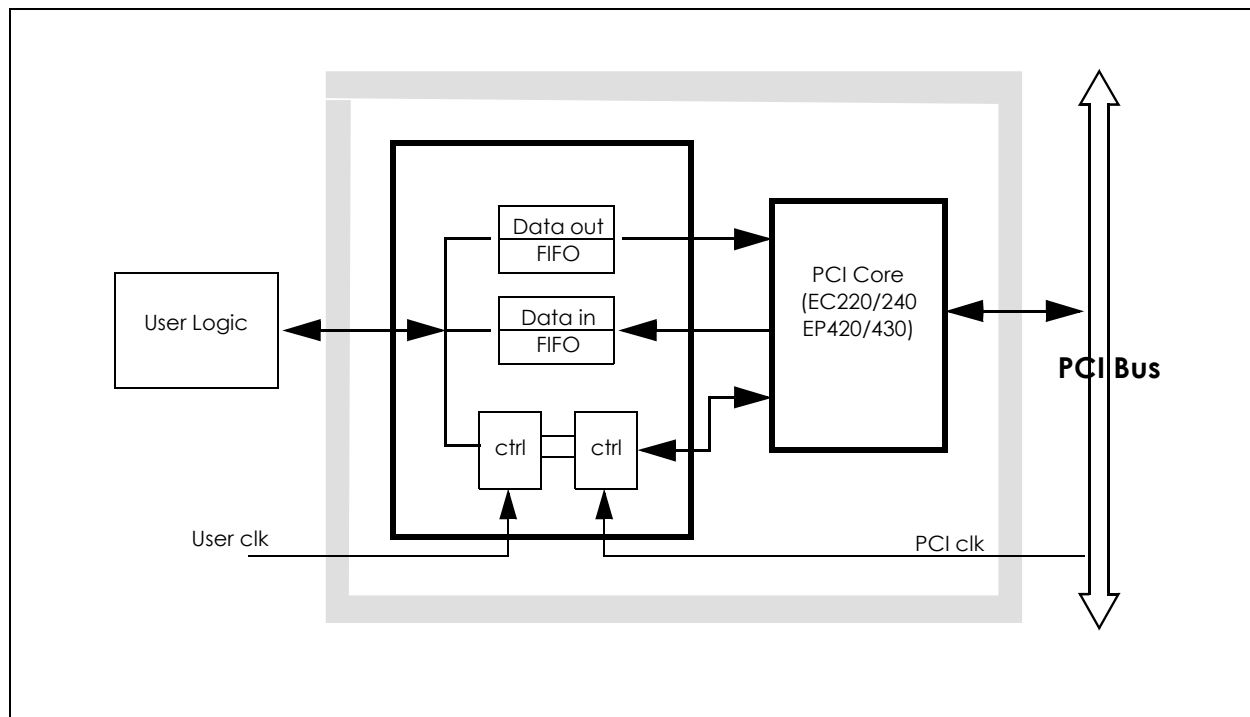


Appendix A. Asynchronous User Interface Option

This appendix describes optional asynchronous user interface for PCI master/target and host bridge cores. This option allows the user logic run at any clock frequencies independent from the PCI clock. This option is implemented as an added-on module located between the PCI core and the user logic. The interface to the PCI core runs at PCI clock speed while the interface to the user logic runs at user logic clock speed.

This option is normally not included in the standard PCI core. User must specify this option when ordering this IP core.

A.1 BLOCK DIAGRAM



A.2 FEATURES

- User interface bus operates asynchronously with PCI bus clock.
- Compatible with PCI master/target and host bridge.
- Double data FIFO provides high speed write posting.
- Independent FIFOs for read operations.
- User-friendly interface.
- Designed for FPGA and ASIC implementations.
- Utilize FPGA built-in dual-port SRAM.
- Supports zero wait state data bursting in user interface and PCI interface.



- Concurrent data transfer on user interface and PCI interface.

A.3 Naming Convention

As far as the user logic is concerned, the PCI core is capable of 4 different types of operations.

- (1) Master read. Initiated by the user logic, the PCI core functions as a master on the PCI bus for a read transfer. Data flows from the PCI bus to the user logic.
- (2) Master write. Initiated by the user logic, the PCI core functions as a master on the PCI bus for a write transfer. Data flows from the user logic to the PCI bus. Both master read and master write are initiated by the user logic. The PCI core becomes a PCI bus master in response to the user logic request to initiate the transfer.
- (3) Target read. Initiated by other device on the PCI bus, the PCI core functions as a target on the PCI bus for a read transfer. Data flows from the user logic to the PCI bus.
- (4) Target write. Initiated by other device on the PCI bus, the PCI core functions as a target on the PCI bus for a write transfer. Data flows from PCI bus to the user logic. Both target read and target write are initiated by another PCI master and the PCI core responds to the request and forwards it to the user logic.

A.4 User Interface Signal Description

Signal name with “_B” or “#” suffixes indicates active low signals.

Table 5: User Interface for Master Operations

Name	Type	Descriptions
U_ADS#	I	Address strobe (master operation). Driven by the user logic to request PCI master operation. When this signal is asserted, the user logic must provide the valid address and control signals on the user interface.
U_ADR[31:0]	I	Address input. Driven by the user logic to the core to indicate the address to be used for PCI master operation.
U_BE#[3:0]	I	Byte Enable. Indicates the active bytes for PCI master read and write cycles. The user interface has 4 byte enables for 32-bit data bus and 8 byte enables for 64-bit data bus.



Table 5: User Interface for Master Operations

Name	Type	Descriptions																																				
U_CMD[1:0]	I	Command Type. This signal is used in conjunction with U_MIO and U_WR to define the PCI command to be issued as a master according to the following table: <table><tr><td>U_CMD</td><td>U_MIO</td><td>U_WR</td><td>PCI Command</td></tr><tr><td>00</td><td>1</td><td>0</td><td>memory read</td></tr><tr><td>01</td><td>1</td><td>0</td><td>memory read multiple</td></tr><tr><td>10</td><td>1</td><td>0</td><td>memory read line</td></tr><tr><td>00</td><td>1</td><td>1</td><td>memory write</td></tr><tr><td>00</td><td>0</td><td>0</td><td>IO read</td></tr><tr><td>01</td><td>0</td><td>0</td><td>Interrupt ack</td></tr><tr><td>00</td><td>0</td><td>1</td><td>IO write</td></tr><tr><td>10</td><td>1</td><td>1</td><td>write invalidate</td></tr></table>	U_CMD	U_MIO	U_WR	PCI Command	00	1	0	memory read	01	1	0	memory read multiple	10	1	0	memory read line	00	1	1	memory write	00	0	0	IO read	01	0	0	Interrupt ack	00	0	1	IO write	10	1	1	write invalidate
U_CMD	U_MIO	U_WR	PCI Command																																			
00	1	0	memory read																																			
01	1	0	memory read multiple																																			
10	1	0	memory read line																																			
00	1	1	memory write																																			
00	0	0	IO read																																			
01	0	0	Interrupt ack																																			
00	0	1	IO write																																			
10	1	1	write invalidate																																			
U_CS#[2:0]	I	Chip select. This signal is available for host bridge designs (EP430 and EP420) only. U_CS# is a 3 bit input to qualify U_ADS# and to select different types of operations to be performed by the master. Bit 0 selects normal PCI memory or I/O operations, bit 1 selects CONFIG_ADDR register access, and bit 2 selects CONFIG_DATA register access.																																				
U_DRD [31:0]/[63:0]	O	Data output. Driven by the core to the user interface. It is used for transferring data during master read operations.																																				
U_DWR [31:0]/[63:0]	I	Master write data input. It transfers data during master write operation. Data bus may be 32-bit or 64-bit, depends on the options specified by user.																																				
U_ERR#	O	Asserted by the core to indicate that the master request cannot be processed. When it is asserted, the master operation is terminated immediately. This signal is used when the core (1) cannot process a master read on PCI due to PCI master or target abort, or (2) cannot process PCI memory or I/O read because the master bit in the configuration register is not enabled. It should be noted that U_ERR# is not asserted for PCI master abort during configuration read access since it is not considered as an error condition.																																				

**Table 5: User Interface for Master Operations**

Name	Type	Descriptions
U_FAST	I	Indicates that the user logic can provide data with zero wait state on master write access AND the user bus clock is faster than the PCI clock. When U_FAST is asserted (high), the PCI core starts the master write operation on the PCI bus as soon as the first data is written into the write buffer. It does not wait until the last data is available.
U_MIO	I	Memory and IO Select. This signal selects between the memory (high) and IO(low) address space.
U_RDNXT#	I	Read next data. This signal is driven by the user logic to read the next data from the FIFO after a PCI master read operation.
U_RDY#	O	This signal is asserted by the core to indicate that it is ready to accept data for master write operation or data is available now for master read operation.
U_SIZE[6:0]	I	U_SIZE indicates the length of the burst transfer, measured in the number of 32-bit word (EC220, EP430) or 64-bit words (EC240, EP420). The exact width of the U_SIZE signal depends on the maximum burst length, which is limited by the FIFO depth of the core. If the maximum burst size is 64 words, U_SIZE has 7 bits. U_SIZE value of “0000001” indicates a single transfer. U_SIZE value “1000000” indicates 64 word transfer. All value between 1 to 64 are allowed. Other values of U_SIZE are illegal.
U_WREN#	I	Asserted by the user logic to write data into the master write buffer.
U_WR	I	This signal is sample with U_ADS# for PCI access. It is generated by the back-end bus controller to indicate the transfer requested to the PCI master/target. It is low for read and high for write.

Table 6: User Interface for Target Operations

Name	Type	Descriptions
T_ADS#	O	Address strobe (target operation). Driven by the core to indicate PCI target access to the core.
T_ADR[31:0]	O	Address output bus. Driven by the core to the user interface during PCI target access. It is the access address on the user logic.
T_BE[3:0]#	O	Byte Enable for target access. The user interface has 4 byte enables for 32-bit data bus and 8 byte enables for 64-bit data bus.

**Table 6: User Interface for Target Operations**

Name	Type	Descriptions
T_BLAST#	O	This signal indicates to the user logic that it is transferring the last data of the current transfer. T_BLAST# is asserted with T_ADS# if the transfer is for single data.
T_CS#	O	Target chip select. Asserted by the core to indicate that the back-end bus is being accessed by another PCI master as a target of the transfer. This signal is asserted for the duration of transfer on the user interface. By monitoring this signal, the user knows when the transfer terminates. If the user specifies multiple base address registers (BAR) within the PCI configuration registers, the core generates multiple T_CS# signals, one for each BAR.
T_DRD [31:0]/[63:0]	I	Data input. Driven by user logic. It is used for transferring data during target read operations. Data bus may be 32-bit or 64-bit, depends on the options specified by user.
T_DWR [31:0]/[63:0]	O	Target write data output. Data driven by the core. It transfers data during target write operation. Data bus may be 32-bit or 64-bit, depends on the options specified by user.
T_RDY#	I	This signal indicates valid data transfer on the user interface for target operations. It means either that data on the T_DWR bus has been accepted for target write or data is available on T_DRD for target read.
T_WR	O	This signal is valid for target access cycle. It is generated by the master/target to indicate if the target access is read (low) or write (high).

Table 7: User Interface Command to Master & Target Operations

Name	Type	Descriptions
SYSCLK	I	User clock. This clock is synchronized with the user logic. All inputs on the user interface are sampled at the rising clock edge of SYSCLK. The PCI clock and SYSCLK can be totally asynchronous to each other and either clock can run at higher or lower speed relative to the other.

**Table 7: User Interface Command to Master & Target Operations**

Name	Type	Descriptions
H_INTA#	I	Interrupt request from the user logic to the PCI bus. This signal is a PCI clock domain signal. The INTA# output on the PCI bus is asserted immediately (after propagation delay) when H_INTA# is asserted.

A.5 Timing Diagrams

Master Write Operation

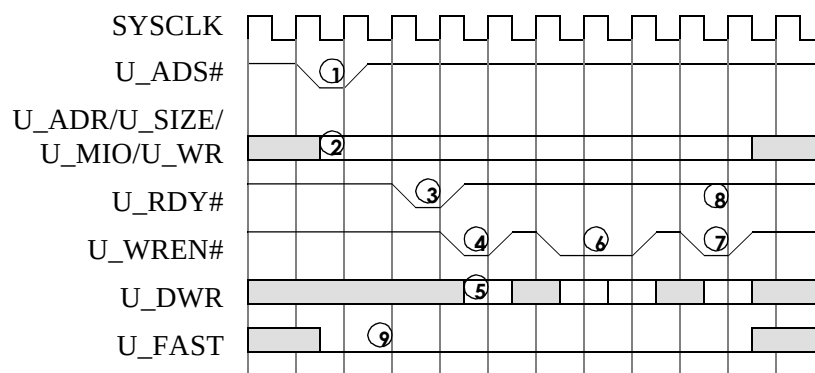


Figure 5 Master write

The above diagram illustrates the concepts of master write by the user logic on the user interface.

- (1) User logic assert U_ADS# for one cycle to indicate a master request.
- (2) The access address and control signals must be valid at the time U_ADS# is asserted. The address and control signals must remain valid until the end of the transfer.
- (3) The PCI core asserts U_RDY# when it is ready to receive write data from the user logic. This signal is asserted for one clock cycle. There may be multiple cycles of delay between time (1) and (3), depending on whether the data FIFO is available at the time of request.
- (4) The user logic asserts U_WREN# when write data is available. U_WREN# can be asserted only after U_RDY# is asserted. Data is written into the FIFO when U_WREN# assertion. For user logics that can always provide write data at zero wait state, U_WREN# can be asserted continuously.



- (5) User logic drives the write data on to U_DWR when U_WREN# is asserted.
- (6) & (7) U_WREN# can be asserted continuously for zero wait state data transfer or asserted with any number of wait state in between successive assertion.
- (8) Data transfer between the user logic and the core ends when the number of write data matches with the U_SIZE value. The user logic can de-assert U_WREN# and stop providing data.
- (9) The U_FAST signal is provided by the user logic to fine-tune the performance of the core. U_FAST indicates that the user logic has higher data bandwidth than the PCI bus. This is possible only if the user logic bus frequency is faster than the PCI bus frequency AND U_WREN# is asserted with zero wait state at all time. If U_FAST is asserted(high), the PCI core starts to transfer data on the PCI bus as soon as possible after the first assertion of the U_WREN# signal. If U_FAST is de-asserted(low), the PCI core starts to transfer data on the PCI bus only after all data has been written into the FIFO.

Double data buffer for master write

The asynchronous option supports dual write buffer for master write posting. Write data can be posted into one write buffer by the user interface while data is extracted from the other buffer to the PCI bus at the same time. The size of the write buffer is determined per application basis. Normally the write buffer size is the same as the cache line size. The following timing diagram shows the operation



of the dual write buffer.

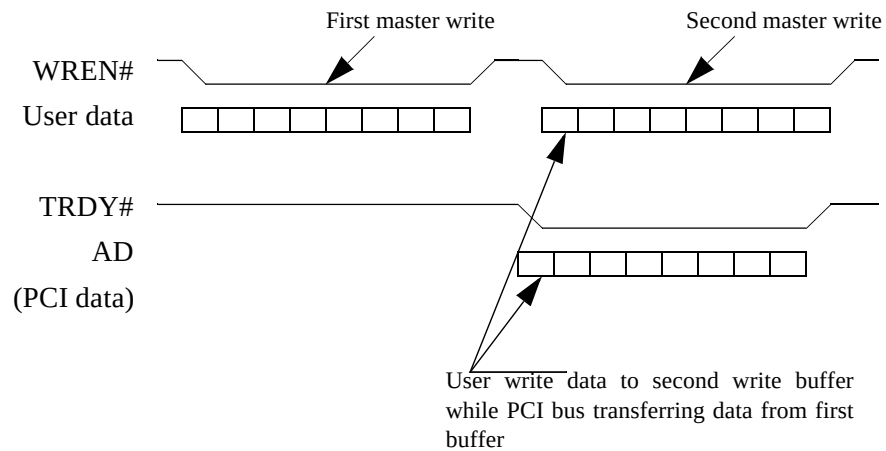


Figure 6 Double write buffer on master write



Master Read Operation

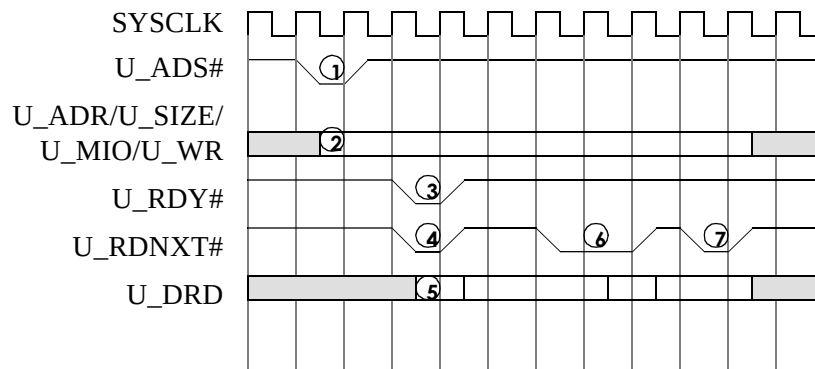


Figure 7 Master read

The above diagram illustrates the concepts of master read by the user logic on the user interface.

- (1) User logic assert U_ADS# for one cycle to indicate a master request.
- (2) The access address and control signals must be valid at the time U_ADS# is asserted. The address and control signals must remain valid until the end of the transfer.
- (3) The PCI starts to read data from the PCI bus as soon as possible after the assertion of U_ADS#. The PCI core asserts U_RDY# when it receives all the read data from the PCI bus.
- (4) The user logic can read the data by asserting the U_RDNXT# signal. This signal indicates that the current data from the PCI core, available on the U_DRD bus, is taken by the user logic and the PCI core should provide the next read data in the following clock cycle.
- (5) Valid data is on the U_DRD bus as soon as U_RDY# is asserted.
- (6) The user can de-assert U_RDNXT# to insert wait states while receiving data from the core. For user logic that can always receives read data with zero wait state, U_RDNXT# can be asserted continuously.
- (7) When the number of data transferred matches with the U_SIZE value, the transfer stops and the user must de-assert U_RDNXT# signal.

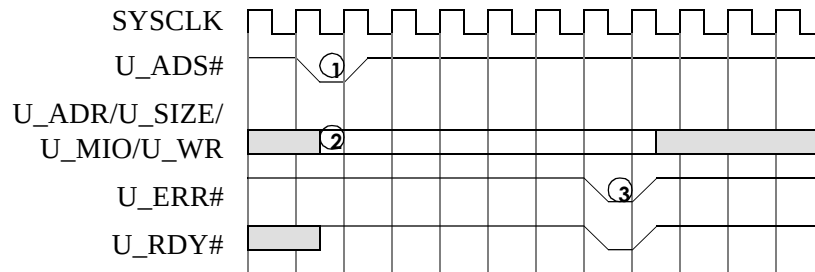
**Master Read Terminated with Error**

Figure 8 Master read terminated with error

The above diagram illustrates a master read terminated with error.

- (1) User logic assert U_ADS# for one cycle to indicate a master request.
- (2) The access address and control signals must be valid at the time U_ADS# is asserted. The address and control signals must remain valid until the end of the transfer.
- (3) The PCI starts to read data from the PCI bus as soon as possible after the assertion of U_ADS#. If the PCI transfer results in master abort, target abort, or if the master enable bit in the PCI configuration register is not set, the read transfer cannot be completed normally and U_ERR# and U_RDY# are both asserted. The transfer on the user interface ends when U_ERR# is asserted.



Multiple Master Operations

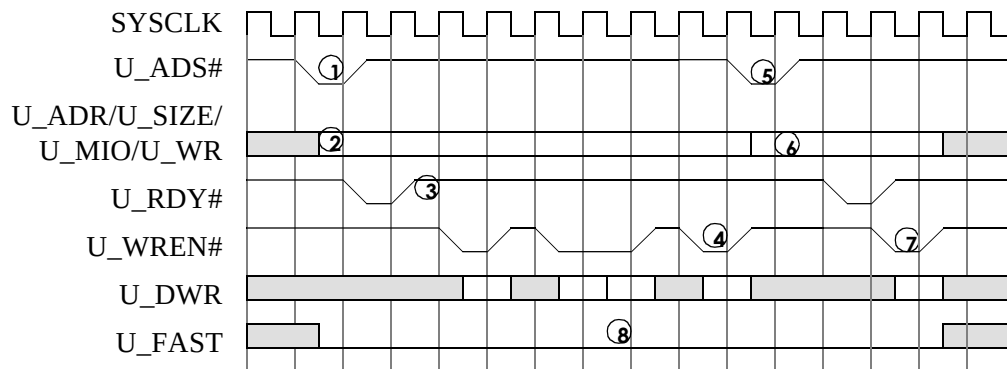


Figure 9 Master write followed by master write

The above diagram illustrates two back-to-back master write operations.

- (1) User logic assert U_ADS# for one cycle to indicate a master request.
- (2) The access address and other control information are provided at the same cycle when U_ADS# is asserted.
- (3) U_RDY# is asserted by the core to start receiving the write data.
- (4) Write data are sampled when U_WREN# is asserted. This example shows a burst write of 4 words so the transfer ends when U_WREN# is asserted for the 4th data.
- (5) The user can assert U_ADS# again to start a second transfer as soon as the previous transfer is completed.
- (6) In the above example the second transfer is a single word write. All the control signals and transfer address must be valid when U_ADS# is asserted.
- (7) U_WREN# can be asserted anytime after U_RDY# is asserted.
- (8) During both transfers U_FAST should remain unchanged. It is expected that U_FAST does not change from transfer to transfer.

Since there are two write buffers to handle master write transfer, the core asserts U_RDY# to accept the write transfer as long as one or more write buffer is available to hold the write data. Up to two write transfers can be posted in the core. When the write transfer is forwarded to the PCI bus, the core guarantee that the write transfers are processed in the same order as they are received from the user interface.

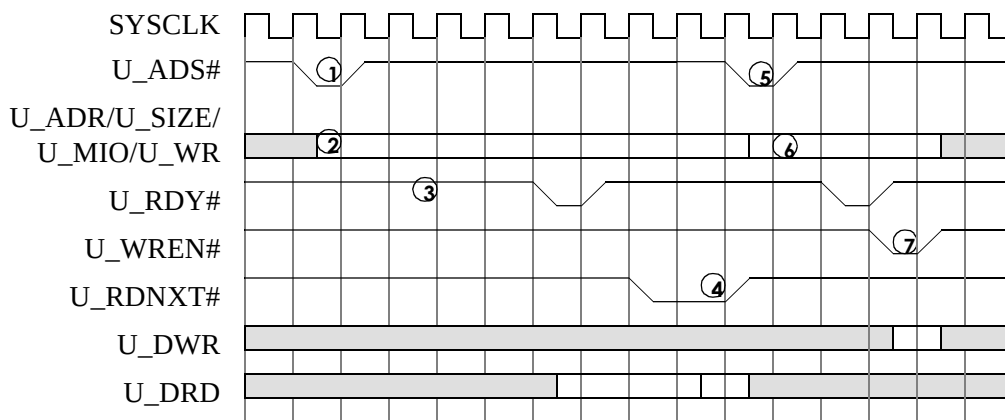


Figure 10 Master read followed by master write

The above diagram illustrates a master read operation followed by a master write operation.

- (1) User logic assert U_ADS# for one cycle to indicate a master request.
- (2) The access address and other control information are provided at the same cycle when U_ADS# is asserted.
- (3) U_RDY# is asserted by the core when all the read data is available from the PCI bus.
- (4) The user logic asserts U_RDNXT# to read the data from the core. This example shows a burst read of 2 words so the transfer ends when U_RDNXT# is asserted for the second time.
- (5) The user can assert U_ADS# again to start a second transfer as soon as the previous transfer is completed.
- (6) In the above example the second transfer is a single word write. All the control signals and transfer address must be valid when U_ADS# is asserted.
- (7) U_WREN# can be asserted anytime after U_RDY# is asserted.



Target Read Operation

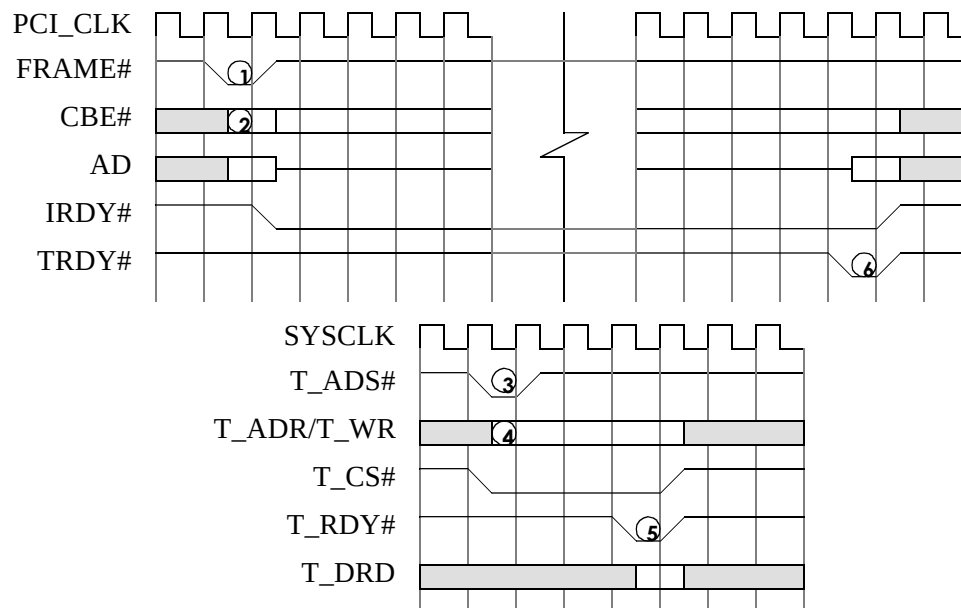


Figure 11 Target read operation

The above diagram illustrates a target read transfer between the PCI and user interface.

- (1) FRAME# is asserted on the PCI bus and the CBE# signal indicates that it is a read operation.
- (2) The access address is valid when FRAME# is asserted. The address is decoded by the PCI target core. If the address is within the address spaced assigned to the core, the PCI core will respond to the request by forwarding the request to the user interface.
- (3) The T_ADS# signal is asserted on the user interface to indicate target access. Since the user interface and the PCI bus are running at different clock, the delay between FRAME# and T_ADS# varies. The PCI core internally synchronized the request signal between the PCI bus and the user interface.
- (4) The request address and other control signals are valid at the time T_ADS# is asserted.
- (5) The user logic asserts T_RDY# signal when the read data is available. The read data is valid on the T_DRD bus and is sampled by the core.



- (6) The ready signal completes the request/acknowledge protocol between the user interface and the PCI bus. After the delay due to synchronization, the read data is driven on the PCI bus. The PCI core signals the availability of data by asserting the TRDY# signal.
- (7) The above diagram shows the sequence of a single read access. Burst read access is very similar to single read, except that both the user interface and the PCI bus has multiple data phase. Each data phase on the user interface is indicated by the T_RDY# assertion and each PCI data phase is indicated by a TRDY# assertion.

Flow-through on burst read

The asynchronous option supports data flow-through automatically on target read. In the event of a burst read, read data is provided to the PCI bus at the same time more data is returned from the back-end. This cuts down the data latency for long burst read operation. The size of the read buffer is determined per application basis. Normally the read buffer size is the same as the cache line size.

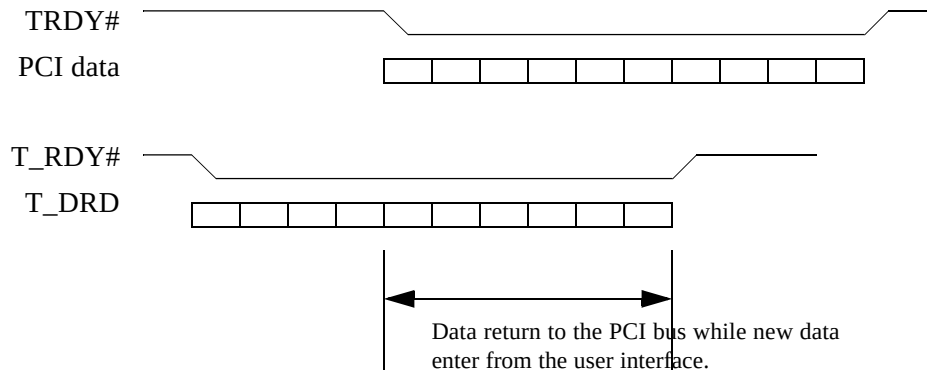


Figure 12 Data flow-through on burst target read



Target Write Operation

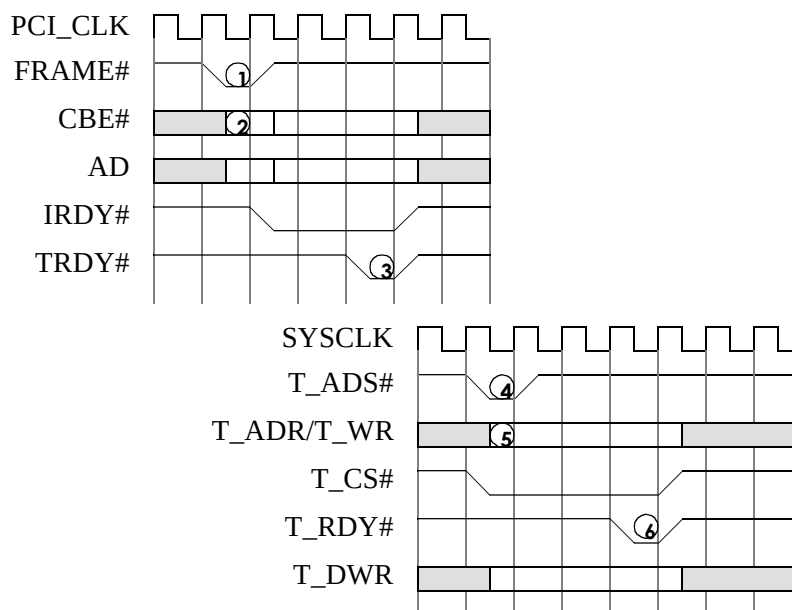


Figure 13 Target write operation

The above diagram illustrates a target write transfer between the PCI and user interface.

- (1) FRAME# is asserted on the PCI bus and the CBE# signal indicates that it is a write operation.
- (2) The access address is valid when FRAME# is asserted. The address is decoded by the PCI target core. If the address is within the address space assigned to the core, the PCI core will respond to the request by forwarding the request to the user interface.
- (3) The write data is posted in the write buffer inside the PCI core immediately. TRDY# is asserted by the PCI core to complete the transfer on the PCI bus.
- (4) After the write data is posted, the T_ADS# signal is asserted on the user interface to indicate target access. Since the user interface and the PCI bus are running at different clock, the delay between FRAME# and T_ADS# varies. The PCI core internally synchronizes the request signal between the PCI bus and the user interface.
- (5) The request address and other control signals are valid at the time T_ADS# is



asserted.

- (6) The user logic asserts T_RDY# signal when write is completed.
- (7) The above diagram shows the sequence of a single write access. Burst write access is very similar to single write, except that both the user interface and the PCI bus has multiple data phase. Multiple data can be posted in the write buffer and multiple data is written on the user interface with a single T_ADS# access.

Double data buffer for target write

The asynchronous option supports dual write buffer for target write posting. Write data can be posted into one write buffer by the PCI bus while data is extracted from the other buffer from the user interface at the same time. The size of the write buffer is determined per application basis. Normally the write buffer size is the same as the cache line size. The following timing diagram shows the operation of the dual write buffer.

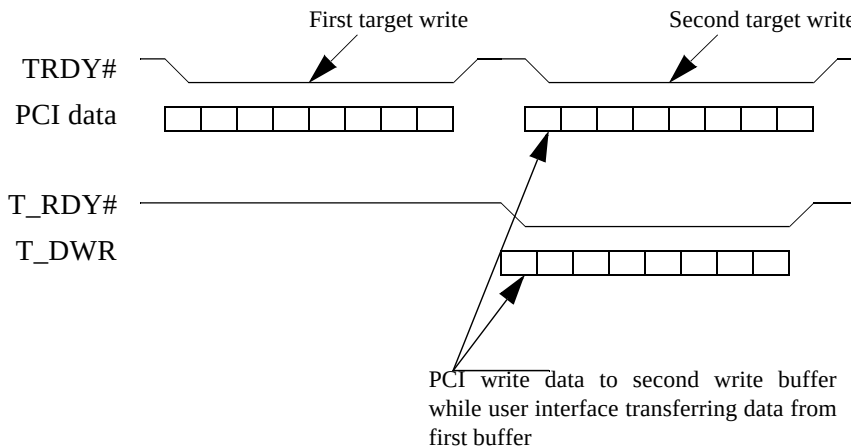


Figure 14 Double write buffer on target write

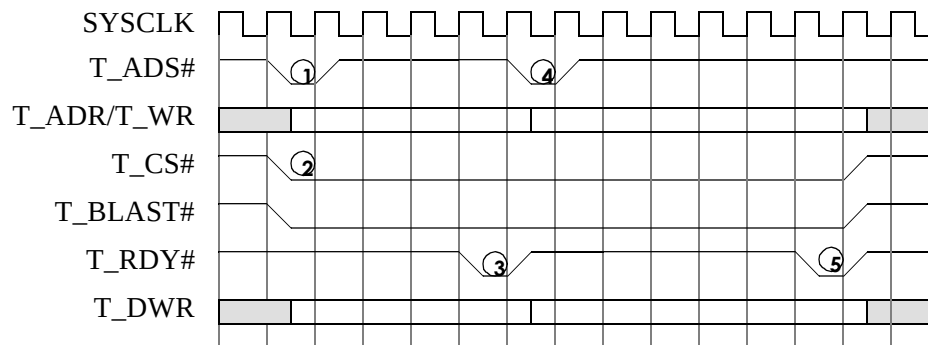
**Back-to-Back Target Access Timing**

Figure 15 Back-to-back access

The above diagram illustrates the timing of back-to-back transfers in the user interface. Target write operation is used to illustrate the timing but the same principle applies to target read.

- (1) When a target write operation is posted in the core, T_ADS# is asserted on the user interface to indicate target access.
- (2) T_CS# is asserted at the same time as T_ADS# and it remains asserted for the duration of the transfer. If it is a single write operation, T_BLAST# is also asserted at the same time to indicate that the last data is being transferred.
- (3) The user logic asserts T_RDY# signal when write is completed.
- (4) The second transfer may start as soon as the previous transfer ends.
- (5) The user logic asserts T_RDY# signal when write is completed.



T_BLAST# Signal Timing

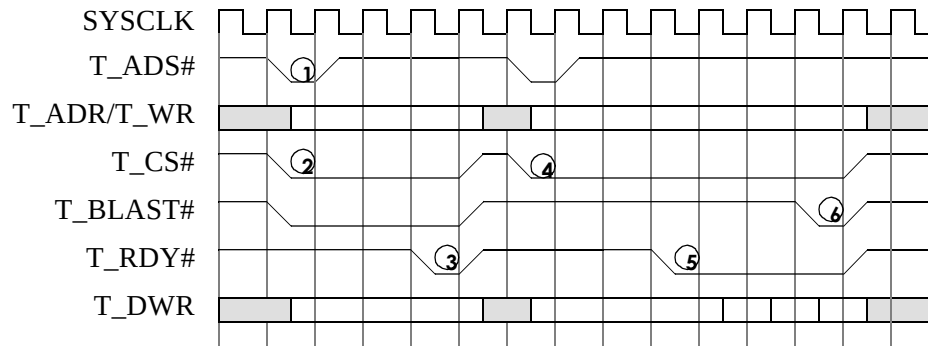


Figure 16 T_BLAST# timing

The above diagram illustrates the timing of the T_BLAST# signal in the user interface. Target write operation is used to illustrate the timing but the same principle applies to target read.

- (1) When a target write operation is posted in the core, T_ADS# is asserted on the user interface to indicate target access.
- (2) T_CS# is asserted at the same time as T_ADS# and it remains asserted for the duration of the transfer. If it is a single write operation, T_BLAST# is also asserted at the same time to indicate that the last data is being transferred.
- (3) The user logic asserts T_RDY# signal when write is completed.
- (4) In this example, the second target write transfer is for 4 words of data. T_BLAST# is not asserted when T_ADS# and T_CS# is asserted.
- (5) The user logic asserts T_RDY# signal when write is completed.
- (6) T_BLAST# is asserted when the last of the 4 word is transferred on the user interface. The transfer ends when both T_BLAST# and T_RDY# are asserted at the same time.



Appendix B. MiniPCI and Power management option

This appendix describes the optional Power Management & Mini-PCI support for PCI cores. The Mini-PCI and Power Management are separate options but typically are used together. The MiniPCI option is application when the EP454 is used as AHB-PCI host bridge or as a peripheral PCI master/target. The Power Management option is normally used only when the EP454 is a peripheral master/target.

Power Management support is compliant to PCI Power Management Interface Specification Rev 1.1. It is an optional features that can be included in Eureka Technology's standard PCI and Mini-PCI core.

The Power Management & Mini-PCI options are implemented by modifications internal to the PCI core, user interface, and its configuration registers. All modifications are include in the core before delivery to customer.

This option is normally not included in the standard PCI core. User must specify this option when ordering this IP core.

B.1 Configuration Register Modifications

PCI configuration register is included in the PCI core. To support power management, the configuration register block is expanded to include additional registers and fields of data. The following table describes the configuration registers with power management. New register locations are highlighted in yellow.

Table 8: PCI Configuration Registers

Device ID		Vendor ID		00h
Status (Bit 4)		Command		04h
Class Code			Revision ID	08h
BIST	Header	Lat. timer	Cache size	0Ch
Base Address Registers 0				10h
Base Address Registers 1				14h
Base Address Registers 2				18h
Base Address Registers 3				1Ch
Base Address Registers 4				20h
Base Address Registers 5				24h
Cardbus CIS pointer				28h
Subsystem ID		Subsystem Vendor ID		2Ch
Expansion ROM				30h
Reserved			Cap_ptr	34h
Reserved				38h

**Table 8: PCI Configuration Registers**

Max_Lat	Min_Gnt	Int. pin	Int. line	3Ch
Power Management Capability (PMC)		Next Ptr	Cap. ID	location determined by Cap_ptr Cap_ptr + 4
Data	PMCSR-BSE Bridge Support extension	Power Management Control/Status Register (PMCSR)		

The following table describes the modified fields in detail.

Table 9: Power Management Modified Fields

Offset	Bit	Name	Access	Description
6	4	Capability	R	This bit is hardwired to “1” to indicate the Power Management capability. Other bits of this register remains unchanged from the standard core.
34(hex)	7:0	Cap_ptr	R	This field provides the starting address of the capability link list. This field is provided by the user as PME_CAP input in the back-end bus. The PCI core uses this field to determine the location of the link list. The last 3 bits of the PME_CAP register must be zero so that the capability link list is properly aligned.
cap_ptr	7:0	Capability ID	R	This field is hardwired to 01h to indicate power management link list
cap_ptr + 1	7:0	Next Item Ptr	R	This field is hardwired to 00h to indicate last of the link list
cap_ptr + 2	15:0	Power Management Capability (PMC)	R	This field provides the operation characteristic of the PCI device. It contains information such as power saving level, current consumption, etc. This field is provided by the user logic as PME_PMC input to the back-end bus.

**Table 9: Power Management Modified Fields**

Offset	Bit	Name	Access	Description
cap_ptr + 4	15	Power Management Command/Status Register	R/W/C	PME Status. This bit is set by the user logic when it intends to assert the PME# signal on the PCI bus. The user asserts the -PME_SET_STATUS input pin on the back-end bus to set this bit. This bit is cleared when a “1” is written by configuration write cycle. For system that does not assert PME# during D3cold state, this bit is reset during PCI reset. For system that supports PME# assertion during D3cold state, this bit is not affected by PCI reset. The core uses bit15 of PME_PMC input to determine if this support is needed. The PME_PMC bit 15 must be valid during PCI reset.
	14:13		R	Data Scale. This field is provided by the user on the back-end bus as the PME_DSC input.
	12:9		R/W	Data Select. This field can be read and write by configuration access, the PME_DSEL output pins on the back-end bus reflects the value of this register.



Table 9: Power Management Modified Fields

Offset	Bit	Name	Access	Description
cap_ptr + 4	8	Power Management Command/Status Register	R/W	PME_EN. This field can be read and write by configuration access, the PME_EN output pins on the back-end bus reflects the value of this register. The PCI PME# signal is asserted when both this bit and the PME_status bit (bit 15) are asserted. For system that does not assert PME# during D3cold state, this bit is reset during PCI reset. For system that supports PME# assertion during D3cold state, this bit is not affected by PCI reset. The core uses bit15 of PME_PMC input to determine if this support is needed. The PME_PMC bit 15 must be valid during PCI reset.
	7:2		R	Reserved, hardwired to zero
	1:0		R/W	PowerState: This field can be read and write by configuration access, the PME_PSOUT output pins on the back-end bus reflects the value of this register written by configuration write. During configuration read, the PME_PSIN input from the back-end bus is returned to the PCI bus.
cap_ptr + 6	7:0	PMCSR-BSE bridge support extension	R	This field is hardwired to “0” since it is not a PCI-to-PCI bridge.
cap_ptr + 7	7:0	Data	R	Read only field provided by the back-end bus as PME_DT input.

B.2 Signal Modifications

The following table shows the signal modifications for power management



option. The “#” suffix in each signal name indicates active low signal. Please note that all signals in the following table operates in PCI clock domain.

Table 10: Signal Modification

Name	Type	Side	Descriptions
CLKRUN#	I/O	PCI	The CLKRUN# signal is used by the system and the PCI card to communicate system clock requirement. CLKRUN# is an open drain output signal from the core.
MPCIACT#	I/O	PCI	Mini-PCI active. This is signal is asserted by the PCI device to indicate that its current function requires full system performance. MPCIACT# is an open drain signal when driven by the core.
PME#	O	PCI	This is an output signal on the PCI bus. It is asserted by the PCI core whenever the PME_EN and PME_status register bits are set to indicate power management event. PME# is an open drain output signal.
H_CRUNI#	I	Back-end	Input signal driven by the user logic to assert the CLKRUN# signal. The core asserts CLKRUN# when H_CRUNI# is asserted.
H_CRUNO#	O	Back-end	Output signal from the core to the user logic. It is a combinational signal reflecting the state of the CLKRUN# signal on the PCI.
H_MPCIACTI#	I	Back-end	Input signal from the back-end requesting assertion of the MPCIACTI# signal. This signal should be asserted by the user logic when it requires full system performance.
H_MPCIACTO#	O	Back-end	Output signal from the core to reflect the value of the MPCIACT# PCI signal.
PME_CAP[7:3]	I	Back-end	This signal is provided by the user to specify the capability pointer value. Bit 2:0 of the pointer is not provided by the user. These three bits are forced to “0” by the core.
PME_DSC[1:0]	I	Back-end	This signal provides the Data Scale input.
PME_DSEL[3:0]	O	Back-end	This output signal reflects the value of the Data Select field in the register.



Table 10: Signal Modification

Name	Type	Side	Descriptions
PME_DT[7:0]	I	Back-end	This signal provides the Data value for configuration read.
PME_EN	O	Back-end	This signal reflects the value of the PME_EN bit in the configuration register.
PME_PMC[15:0]	I	Back-end	This signals provides the PMC values for configuration read.
PME_PSIN[1:0]	I	Back-end	This signal provides the PowerState value for configuration read.
PME_PSOUT[1:0]	O	Back-end	This signal reflects the PowerState value written by configuration write cycle.
PME_SET_STATUS	I	Back-end	This signal is asserted by the user to set the PME Status bit in the configuration register. The PCI core asserts the PME# output when the PME status and PME_EN bits are both set.

B.3 Functional Modifications

Power management support mainly involved in two areas: additional PCI signals and PCI configuration registers. The additional PCI signals are CLKRUN# and PME#. These two signals are controlled by the PCI core based on signals provided by the back-end bus.

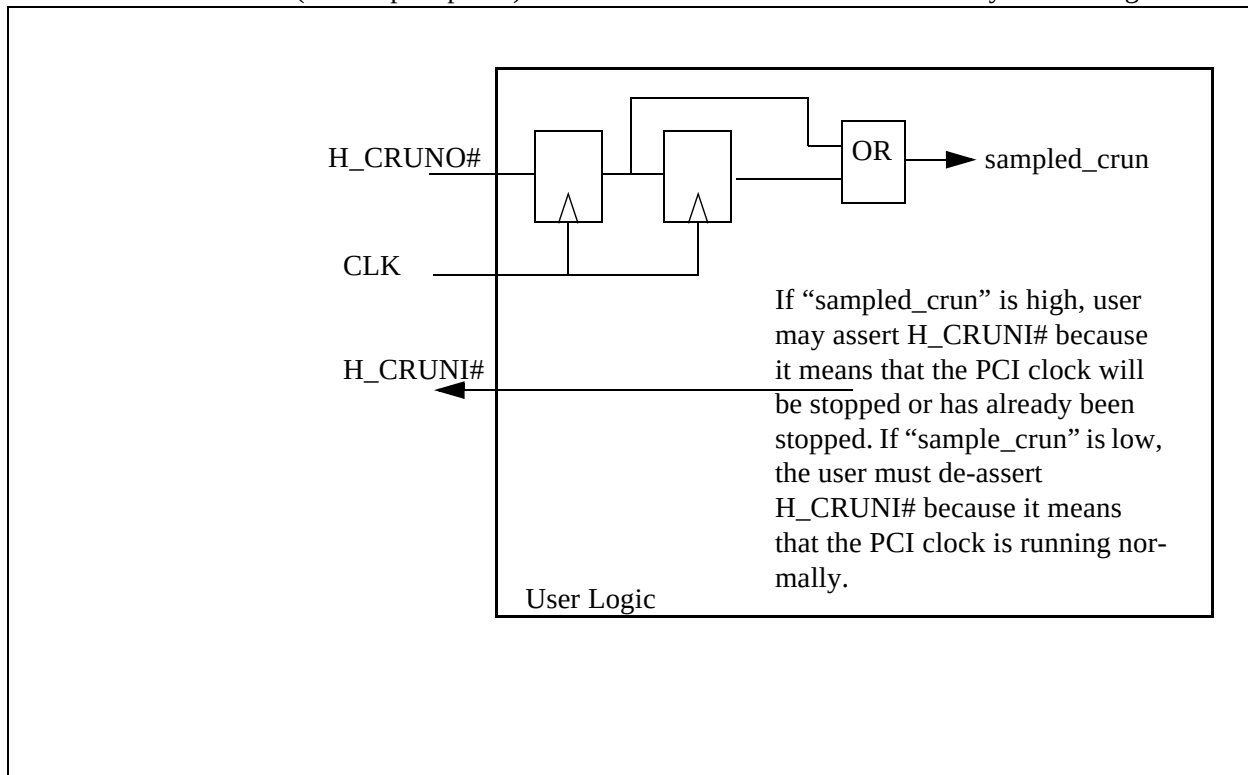
CLKRUN#

CLKRUN# is a bi-directional signal on the PCI bus. CLKRUN# is asserted by the system clock driver when the bus clock is running normally. It is de-asserted four cycles before the system clock is stopped or slowed down to inform all the PCI devices of the clock speed changes. Output from the core is controlled by the H_CRUNI# signal from the back-end. The user asserts the H_CRUNI# signals whenever it wants to inform the system to maintain the bus clock at the normal rate. The PCI core assert CLKRUN# on the PCI bus when H_CRUNI# is asserted. The H_CRUNO# output on the back-end bus reflects the value of the CLKRUN# signal and it can be used by the user logic.

The user logic is required to monitor H_CRUNO# before it asserts H_CRUNI#. It should sample H_CRUNO# by two flip-flops in series inside the user logic code. If the output of either flip-flop is high, it indicates that the system is about to stop



the clock or the clock has already been stopped. If the user wants to restart the clock, it should assert H_CRUNI# to request the system clock generator to start the clock. The PCI core asserts CLKRUN# when H_CRUNI# is asserted. Once the system clock is resumed, the flip-flop that samples H_CRUNO# in the user logic will sample H_CRUNO# and generate a “0” at its output. The user logic should de-assert H_CRUNI# after H_CRUNO# is asserted for two clock cycles (both flip-flop low) so the CLKRUN# will be maintained by the clock generator.

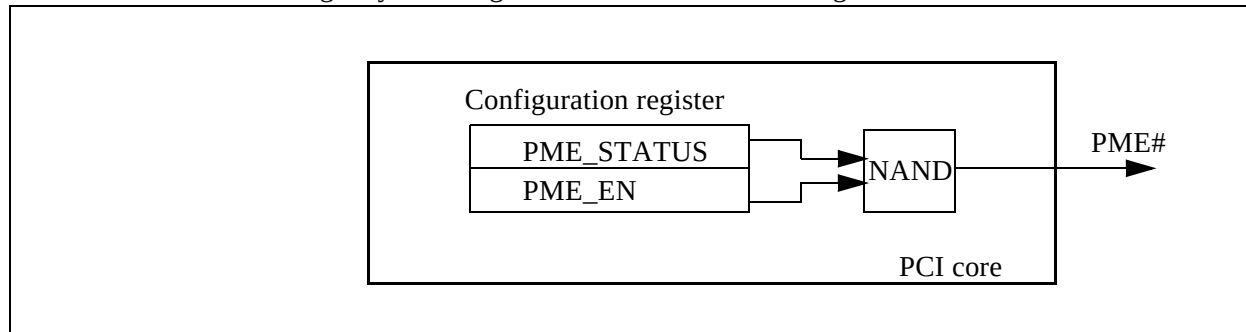


PME#

PME# is an open drain output from the core to the PCI bus. This signal is asserted by the core to request a change in its power management state or to report a power management event. It is asserted by the core when the PME Enable bit and the PME Status bit in the configuration register are both set. The PME Enable bit is set by software through configuration write. The PME Status bit is set by the user



logic by asserting the PME_SET_STATUS signal.



MPCIACT#

MPCIACT# is an open drain output from the core to the PCI bus. This signal is asserted by the core to indicate that its current function requires full system performance or requires that the system not enter any suspended state. It is asserted when the back-end H_MPCIACT# signal is asserted. Internal operation of the core is not affected by the MPCIACT# signal.

Configuration Register

Power management support requires additional configuration registers to be implemented by the core. The additional configuration registers are described in previous section of this document. These registers can be divided into three types: (1) read only register with read value hardwired inside the PCI core, (2) read only registers with read value provided by the back-end bus, and (3) registers which can be written by configuration write or by the back-end bus.

Type 1 register are handled by the core internally. They include the Capability Identifier, Next Item Pointer, PMCSR-BSE and all the reserved fields.

Type 2 registers require input value from the user side. The information required are very standard such as location of the link list or the power consumption level of the device. They include the Capability Pointer, PMC, PowerState(read), Data Scale and the Data field.

Type 3 registers can be written by software during configuration write to inform the back-end of certain system events, or by the back-end to assert the PME# signal. If they are writable by software, the software write value are output to the back-end. They include Data Select, PME_EN, and PowerState(write). The PME status register is the only the PME register that can be set by the back-end. It is set by the PME_SET_STATUS input to assert PME#.

D3cold state support

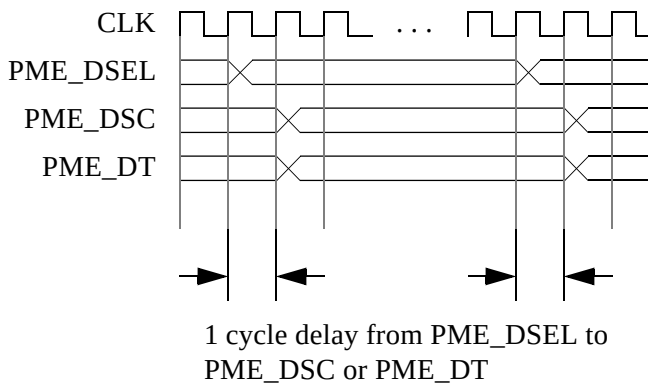
D3cold is one of the power levels allowed by PCI power management. If the core can assert PME# during D3cold state, it must retain the PME_STATUS and PME_EN register bit values during D3cold state. The user must continue to apply power to the core during D3cold state in order to maintain the logic level of these



register bits. Furthermore, when PME# is supported during D3cold state, the PME_STATUS and PME_EN register bits are not changed by PCI reset. If PME# during D3cold state is not supported, the core resets the PME_STATUS and PME_EN registers during PCI reset. The core uses bit 15 of PME_PMC input to determine the whether to reset these bits. The PME_PMC signals must be valid during PCI reset.

PME_DSEL, PME_DSC, PME_DT timing

PME_DSC and PME_DT are input data from the user logic. They are generated according the PME_DSEL output signal from the core. The following diagram shows the timing relationship between PME_DSEL to PME_DSC and PME_DT





Appendix C. 32-bit only Cardbus option for peripheral device

This appendix describes optional CardBus support for PCI master and target cores. This option is application only when the EP454 is used as a peripheral device instead of a host bridge core. When the EP454 is used as a host bridge core, please refer to the 32-bit only or 32/16-bit Cardbus option for host bridge. They are described in other sections of this document.

CardBus support is an optional features that can be included in standard Eureka Technology's PCI cores. CardBus architecture is modeled after the PCI bus architecture. With this CardBus support option in place, the PCI cores from Eureka Technology can be used in CardBus system environments as a CardBus PC Card master or target. All the features of the CardBus system, with the exception of LOCK, are supported by this option.

This option is implemented by modifications internal to the PCI core and its configuration registers, and by addition of CardBus specific signals to the core on the CardBus side and back-end interface.

This option is normally not included in the standard PCI core. User must specify this option when ordering this IP core.

C.1 CardBus Signal Modifications

CardBus and PCI bus shares many similar signals, even though they have different names. The following table shows the signal names when the CardBus option is implemented and their counterparts in the PCI bus environment.

Table 11: CardBus Bus Interface Signals

Name		Type	Descriptions
CardBus	PCI		
CAD[31:0]	AD[31:0]	I/O	Address and Data are multiplexed on the same pins. A bus transaction consists of an address phase followed by one or more data phases. The core supports both read and write bursts.
CC/BE[3:0]#	C/BE[3:0]#	I/O	Bus Command and Byte Enables. They are multiplexed on the same pins. During the address phase of a transaction, C/BE# define the bus command. During the data phase, C/BE# define byte enables.
CCLK	CLK	I	System clock

**Table 11: CardBus Bus Interface Signals**

Name		Type	Descriptions
CardBus	PCI		
CCLKRUN#	--	I/O	This is a CardBus only signal. It is implemented with the CardBus option. The CCLKRUN# signal is used by portable CardBus devices to request the system to turn on the bus clock.
CDEVSEL#	DEVSEL#	I/O	Device Select. When actively driven, indicates the driving device has decoded its address as the target of the current access. As an input, DEVSEL# indicates whether any device on the bus has been selected.
CFRAME#	FRAME#	I/O	Cycle Frame. It is driven by the master/target to indicate the beginning and duration of an access. FRAME# is asserted to indicate a bus transaction is beginning. While FRAME# is asserted, data transfers continue. When FRAME# is de-asserted, the transaction is in the final data phase.
CGNT#	GNT#	I	Grant. It indicates to the master/target that access to the CardBus/PCI bus has been granted.
---	IDSEL	I	Initialization Device Select. It is used as a chip select during configuration read and write transactions. This signal is connected externally to one of the AD signals according to PCI device number. This signal is used only in PCI core and does not exist in CardBus core.
CINT#	INTA#	O	CardBus/PCI interrupt signal. Controlled by back-end bus interrupt request.
CIRDY#	IRDY#	I/O	Initiator Ready. It indicates the master/target's ability to complete the current data phase of the transaction. IRDY# is used in conjunction with TRDY# to indicate a completion of one data transfer.

**Table 11: CardBus Bus Interface Signals**

Name		Type	Descriptions
CardBus	PCI		
CPAR	PAR	I/O	Parity. It is an even parity across AD[31:0] and C/BE[3:0]# signals. It is driven by the master/target for address and write data phases, and is driven by the target for all read data phases.
CPERR#	PERR#	I/O	Parity Error. It is driven by the device on the PCI bus that receives data to report an error condition.
CREQ#	REQ#	O	CardBus/PCI Bus Request. Driven by the master/target to indicate to the arbiter that it needs to use the bus.
CRST#	RESET#	I	System Reset.
CSERR#	SERR#	O	System Error. It is driven by the device on the CardBus/PCI bus that receives address parity errors, data parity errors on the special cycle command, or any other system error.
CSTOP#	STOP#	I/O	Stop indicates the current target is requesting the master/target to stop the current transaction due to unusual conditions.
CTRDY#	TRDY#	I/O	Target Ready. Indicates the target's ability to complete the current data phase of the transaction. It is used in conjunction with IRDY#.

In addition to signals listed in table 1, there are other CardBus signals which are not generated by the core. These signals are either static signals or signals which should be generated directly by the user logic on the back-end bus.

Table 12: CardBus Static/Direct Interface Signals

Signal	Type	Descriptions
CCD[2:1]# CVS[2:1]#	Static	These 4 signals are static signals and they should be hard-wired on the PC board. They are not connected to the core nor the chip where the core resides.

**Table 12: CardBus Static/Direct Interface Signals**

Signal	Type	Descriptions
CAUDIO	O	This is an audio signal that can be used to carry a single amplitude digital waveform or pulse width modulated audio information. This signal, if used, should be generated by the user logic and connected directly to the CardBus connector.
CSTSCHG#	O	This is an optional signal that can be asserted by the CardBus device to signal status change. This signal, if used at all, should be generated by the user logic and connected directly to the CardBus connector.

C.2 Back-end Interface Modification

The optional CardBus supports affects the back-end interface by adding two signals to the back-end. Other back-end interface signals of the PCI core remain unchanged.

Table 13: Additional Back-end Signals

Signal	Type	Descriptions
H_CRUNI#	I	Input signal driven by the user logic to assert the CCLKRUN# CardBus signal. The core samples H_CRUNI# at the rising clock edge and drives CCLKRUN# in the following clock cycle.
H_CRUNO#	O	Output signal from the core to the user logic. It is a combinational signal reflecting the state of the CCLKRUN# signal on the CardBus.

C.3 Configuration Register Modification

With the CardBus option, the PCI core supports a common set of the configuration registers between CardBus and PCI bus. All the PCI bus configuration registers are implemented together with the CardBus specific CIS pointer. The



following is a map of the configuration register space.

Device ID		Vendor ID		00h
Status		Command		04h
Class Code			Revision ID	08h
BIST	Header	Lat. timer	Cache size	0Ch
Base Address Registers 0				10h
Base Address Registers 1 (as requested)				14h
Base Address Registers 2 (as requested)				18h
Base Address Registers 3 (as requested)				1Ch
Base Address Registers 4 (as requested)				20h
Base Address Registers 5 (as requested)				24h
Cardbus CIS pointer				28h
Subsystem ID		Subsystem Vendor ID		2Ch
Expansion ROM				30h
Reserved				34h
Reserved				38h
Max_Lat	Min_Gnt	Int. pin	Int. line	3Ch

The CIS pointer is a read only register and has the following format:

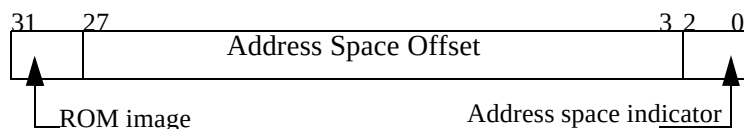


Table 14: CIS Pointer Register Bit Assignment

Bits	Definition
31:28	Rom Image number. Defines the ROM image number in which the CIS is located. The offset is added to the start of the ROM image to identify the start of the CIS.

**Table 14: CIS Pointer Register Bit Assignment**

Bits	Definition
27:3	Address Space Offset. Bit [2:0] define which space the CIS resides within. The address space offset is interpreted as follows for each type of address space: Configuration space - the offset is from location 40h in device specific region up to a maximum value of F8h. Memory space - the offset is from the base address specified by the selected base address register. Expansion ROM space - the offset value is from the start of the ROM image identified by bits[31:28].
2:0	Address Space Indicator. Specifies the base address within the space indicated. The offset (defined by bits[27:3]) is added to this base address to identify the start of the CIS. The address indicators values are: 0 = CIS is device-specific configuration space. 1 = CIS is in the memory pointed to by base address register 0 2 = CIS is in the memory pointed to by base address register 1 3 = CIS is in the memory pointed to by base address register 2 4 = CIS is in the memory pointed to by base address register 3 5 = CIS is in the memory pointed to by base address register 4 6 = CIS is in the memory pointed to by base address register 5 7 = CIS is in the device's expansion ROM. Bits[31:28] identify which ROM image.



Appendix D. Hot -Swap Option for peripheral device

This appendix describes the optional PCI standard Hot-Swap support for PCI master and target cores typically used in Compact PCI environment. It is an optional features that can be included in Eureka Technology's standard PCI cores. For the EP454, this option is applicable only if the device is used as a peripheral master/target. This options is not applicable for a host bridge.

The Hot-Swap support option is implemented by modifications internal to the PCI core, back-end interface, and its configuration registers. All modifications are included in the core before delivery to customer.

This option is normally not included in the standard PCI core. User must specify this option when ordering this IP core.

D.1 Configuration Register Modifications

PCI configuration register is included in the PCI core. To support PCI Hot-Swap, the configuration register block is expanded to include additional registers and fields of data. The following table describes the configuration registers with power management. New register locations are highlighted in yellow.

Table 15: PCI Configuration Registers

Device ID		Vendor ID		00h
Status		Command		04h
Class Code			Revision ID	08h
BIST	Header	Lat. timer	Cache size	0Ch
Base Address Registers 0				10h
Base Address Registers 1				14h
Base Address Registers 2				18h
Base Address Registers 3				1Ch
Base Address Registers 4				20h
Base Address Registers 5				24h
Cardbus CIS pointer				28h
Subsystem ID		Subsystem Vendor ID		2Ch
Expansion ROM				30h
Reserved			cap_ptr	34h
Reserved				38h
Max_Lat	Min_Gnt	Int. pin	Int. line	3Ch
Reserved				Others

**Table 15: PCI Configuration Registers**

	Hot Swap Control Register	Next Pointer	Capability Identification Register	Cap_ptr
--	---------------------------	--------------	------------------------------------	---------

The following table describes the modified fields in detail.

Table 16: Power Management Modified Fields

Offset	Bit	Name	Access	Description
34(hex)	7:0	Cap_ptr	R	This field provides the starting address of the capability link list. This field is provided by the user as HS_CAP input in the back-end bus. The PCI core uses this field to determine the location of the capability list. The last 3 bits of the HS_CAP register must be zero so that the capability link list is properly aligned.
cap_ptr	7:0	Capability ID	R	This field is hardwired to 06h to indicate Hot-Swap capable
cap_ptr + 1	7:0	Next Item Ptr	R	This field is hardwired to 00h to indicate last of the link list



Table 16: Power Management Modified Fields

Offset	Bit	Name	Access	Description
cap_ptr + 2	0	Reserved	R	Read only as '0'.
	1	ENUM_MASK	R/W	ENUM# mask. If this bit is "0", the core asserts ENUM# when REM_STAT or INS_STAT is one. This indicates an insertion or removal event. If this bit is "1", the core does not assert ENUM#
	2	Reserved	R	Read only as "0"
	3	LED	R/W	LED On/Off. This register bit can be read or written by software. The value of this register is reflected on the output signal LED_OUT.
	4:5	Reserved	R	Read only as "00"
	6	REM_STAT	R/C	Signal ENUM# removal event. The core sets this bit when L_STAT input is high and RESET_B is de-asserted, signaling an impending removal. This bit is cleared when a "1" is written by software. Writing zero has no effect on this bit. The value of this bit is reflected on the REM_STAT output from the core.
	7	INS_STAT	R/C	Signal ENUM# insertion event. The core sets this bit when L_STAT input is low and S_PRELD input is high. S_PRELD can be used to indicate completion of serial preload. This bit is cleared when a "1" is written by software. Writing zero has no effect on this bit. The value of this bit is reflected on the INS_STAT output from the core.



D.2 Signal Modifications

The following table shows the signal modifications for Hot-Swap option. The “#” suffix in each signal name indicates active low signal.

Table 17: Signal Modification

Name	Type	Side	Descriptions
ENUM#	O	PCI	This signal indicates a Hot-Swap event. It is controlled by the ENUM_MASK, REM_STAT, and INS_STAT bits in the Hot-Swap configuration register.
HS_CAP[7:3]	I	Back-end	This signal is provided by the user to specify the capability pointer value. Bit 2:0 of the pointer is not provided by the user. These three bits are forced to “0” by the core.
INS_STAT	O	Back-end	Output signal reflecting the value of the INS_STAT bit in the Hot-Swap configuration register.
L_STAT_IN	I	Back-end	Hot-Swap local status input. This signal indicates the state of the ejector switch and is used by the core to control the INS_STAT and REM_STAT Hot-Swap configuration registers.
LED_OUT	O	Back-end	Output signal reflecting the value of the LED bit in the Hot-Swap configuration register.
REM_STAT	O	Back-end	Output signal reflecting the value of the REM_STAT bit in the Hot-Swap configuration register.
S_PRELD	I	Back-end	This signal indicates the completion of serial pre-load completion, indicating that the core is ready for host initialization. It is used by the core to control the INS_STAT bit in the Hot-Swap configuration register.

D.3 Functional Modifications

Hot-Swap support mainly involved PCI configuration register modification to support the additional registers that make up the capability list. Additional signals are provided in the PCI interface and the back-end interface to work with these



registers.

HS_CAP

HS_CAP is a static value provided by the back-end interface to indicate the location of the capability pointer. The capability pointer can be located anywhere outside the first 64 byte of the configuration space so bit[7:6] of the HS_CAP must not be “00”. Furthermore, to facilitate implementation, the capability list must be located at 8-byte boundaries. The 3 least significant bits are assumed to be “000”. The back-end interface only provides HS_CAP[7:3].

ENUM#

ENUM# is an open drain output from the core to the PCI bus. This signal is asserted by the core to indicate an insertion or removal event. This signal can be mask by the ENUM_MASK bit in the configuration register. The generation of this register is based on the REM_STAT and INS_STAT bits in the configuration register. Please see the configuration register section to see the rules governing the assertion of these bits.

Output signals

The value of the LED, REM_STAT and INS_STAT are driven by the core to the back-end interface. User logic can use these signals to control the LED or other functions as required.

Input signals

The value of the L_STAT_IN and S_PRELD signals are input to the core to control the Hot-Swap control registers. The L_STAT_IN is the input side of the L_STAT bi-directional signal. The S_PRELD signal indicates the serial preload completion of the device. Please see the configuration register section to see the usage of these bits by the core.



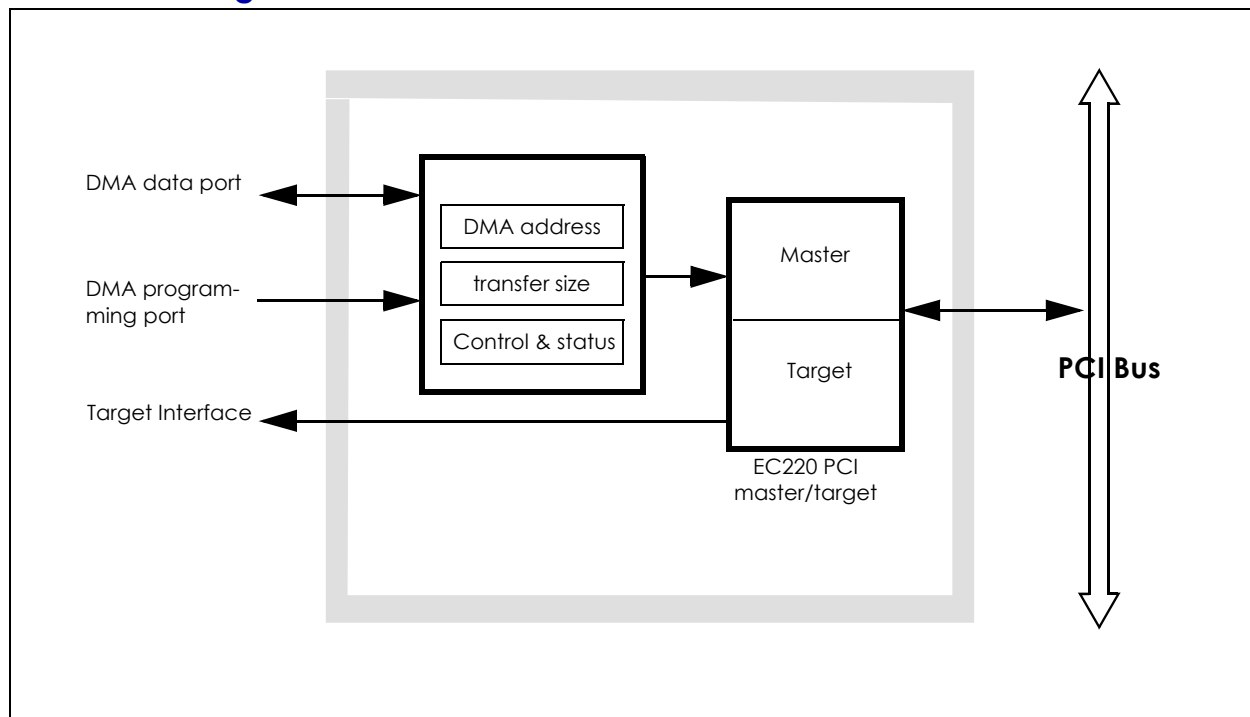
Appendix E. Integrated DMA Controller

This appendix describes optional DMA function for the EC220 or similar PCI master/target or bridge cores. The standard PCI master/target core allows the user to access the PCI bus directly by specifying request by using the user interface signals. This method provides fast access to the PCI bus in real time. The DMA options allows the user to program the DMA registers and then lets the DMA controller to take over the transfer. This method in general requires less interaction between the PCI core and the user logic.

The DMA option does not affect the target operation of the PCI master/target core. The target function, however, can be used to program the DMA controller register if desire. This will be explained in later sections.

The following diagram illustrates the DMA and PCI master/target functions.

E.1 Block Diagram



E.2 Programming Model

The DMA function is controlled by software which program several control registers inside the DMA controller. The following table illustrates the control regis-



ters.

Table 18: DMA Control Registers

Offset	Bits	Descriptions
DMA address register		
00	31:2	Starting PCI address
	1:0	These two bits are hardwired to zero because DMA operates at size of 32-bit words
Transfer size register		
01	31:21	Hardwired to zero
	20:0	The number of 32-bit words to be transferred. The maximum is 2Meg-1.
DMA status register		
10	31:4	Hardwired to zero
	3	DMA direction. If this bit is “0”, it indicates DMA transfer data from PCI to user port. If it is “1”, data is transferred from user port to PCI.
	2	Start Bit. DMA starts when user writes a “1” to this bit. This bit is reset to “0” when DMA finishes
	1	Interrupt bit. This bit is set to “1” when DMA completes. It can be used to trigger interrupt at the completion of DMA. This bit is cleared when user writes a “1” to it. Writing “0” has no effect.
	0	Error bit. This bit is set to “1” when DMA is terminated due to error on the PCI bus. This bit is cleared when user writes a “1” to it. Writing “0” has no effect.

E.3 User Interface Signals

The DMA option replaces the master side of the user interface by DMA programming port and data port. The following table lists the new signals. Signals with



“#” suffix are active low signals.

Table 19: DMA Programming Port

Signal	Type	Descriptions
CR_ADDR[1:0]	I	DMA control register address.
CR_ADS#	I	Access to DMA control registers.
CR_BE#[3:0]	I	Byte enable to DMA control registers.
CR_DRD[31:0]	O	Read data from DMA control registers.
CR_DWR[31:0]	I	Write data to DMA control registers.
CR_RDY#	O	Ready signal to indicate completion of DMA control register access.
CR_WR	I	Read write indicator. If this signal is “1”, the DMA control register is written to, otherwise the DMA control register is read.

Table 20: DMA Data Port

Signal	Type	Descriptions
D_CNT[20:0]	O	Data count. This value is incremented by 1 for each data transferred. At the first data transfer, this value is zero.
D_DRD[31:0]	I	Data transferred from user port to PCI bus.
D_DWR[31:0]	O	Data transferred from PCI bus to user port.
D_ERR#	O	This signal has the opposite value of the error bit in the status register.
D_INTR#	O	This signal has the opposite value of the interrupt bit in the status register. It can be used to trigger interrupt.
D_RDY_RD#	O	This signal is asserted for each data read from D_DRD to be transferred to the PCI bus. The current data on D_DRD is captured and the next data should be on the D_DRD bus in the following cycle.



Table 20: DMA Data Port

Signal	Type	Descriptions
D_RDY_WR#	O	This signal is asserted for each data written to D_DWR which is received from the PCI bus. The user logic must capture the D_DWR data when D_RDY_WR# is asserted.
D_START	O	This signal has the same value as the start bit of the status register.
D_WR	O	This signal has the same value as the direction bit of the status register.

E.4 Accessing DMA Registers

The CR_ADS# signal is asserted by the user logic to indicate read or write to the control registers. CR_WR must be high to signal register write. If CR_WR is low when CR_ADS# is asserted, it is a read access. CR_WR will remain valid until the end of read or write. The address of the control register is presented at the same time and remains valid until the write is completed. Write data must be valid one cycle after CR_ADS# and remains valid until the end of the transfer. Write completion is indicated by the CR_RDY# signal asserted by the control register. When CR_RDY# is asserted, write to the control register is completed.

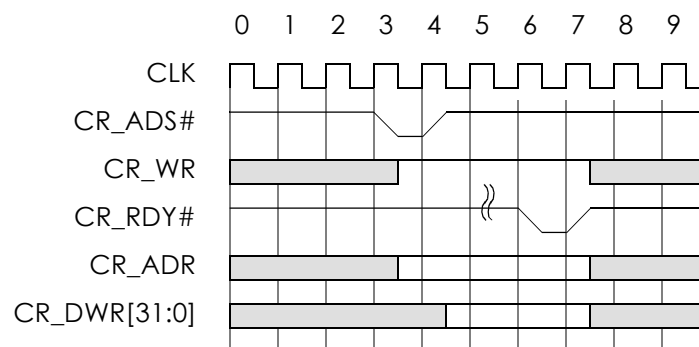


Figure 16.1 Control register write



Similar timing sequence is used for control register read except that CR_WR signal is low.

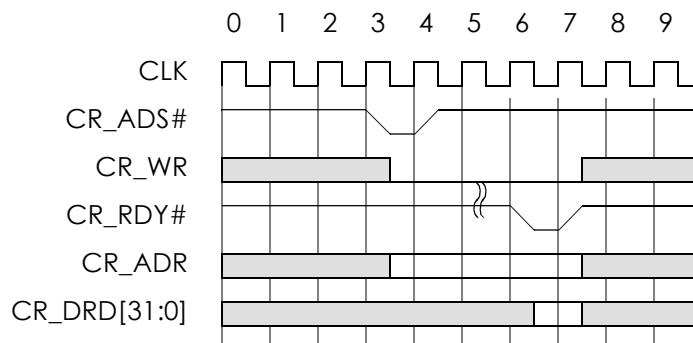


Figure 16.2 Control register read

E.5 DMA Data Transfer

To program a DMA transfer, the user should first program the DMA address and transfer registers. The DMA can then be started by writing a “1” to the start bit in the status register with the proper direction bit.

Once DMA is started, the PCI bus master would start to read or write data from the PCI bus and transfer data to the DMA data port. The PCI bus master would use burst transfer as long as possible to transfer all the DMA data. It is expected the length of the transfer to be limited by the latency timer in the PCI configuration register.

The following diagram illustrates DMA transfer of five 32-bit words from PCI to DMA data port. Please note that the timing of the D_RDY_WR# signal controlled by the speed of the PCI bus. It may be asserted with idle cycles (wait states) in



between data or it can be asserted multiple cycles in a row for consecutive data.

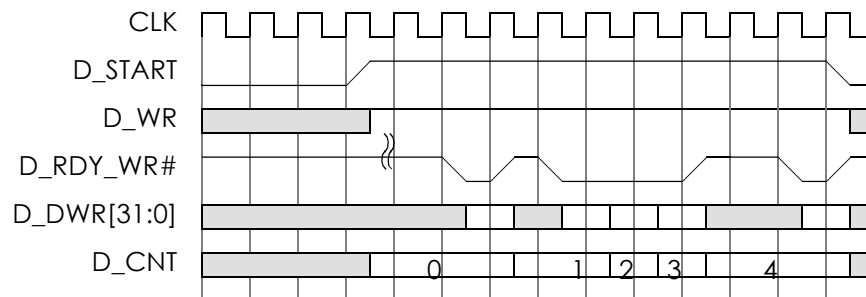


Figure 16.3 DMA from PCI to user port

The following diagram illustrates DMA transfer of five 32-bit words from user port to PCI. Please note that the timing of the D_RDY_RD# signal controlled by the speed of the PCI bus. It may be asserted with idle cycles (wait states) in between data or it can be asserted multiple cycles in a row for consecutive data. The first data must be presented at the D_DRD bus when the start bit is asserted and remains valid until D_RDY_RD# is asserted. Except for the last data of the transfer, the next data must be valid at the D_DRD bus one cycle after the asser-



tion of the D_RDY_RD# signal.

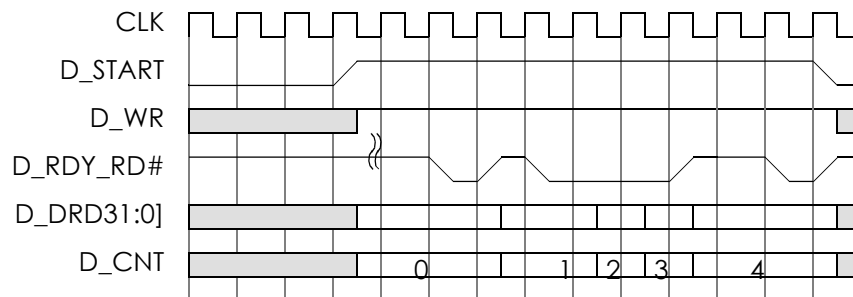


Figure 16.4 DMA from user port to PCI

E.6 Program the DMA from the PCI Bus

It is possible to program the DMA from the PCI bus by making use of the PCI target port. This can be achieved without changing the core at all. The following signals can be connected together to give control of the DMA programming port to the PCI bus.

Table 21: DMA Programming Port

DMA programing port	Target side user interface
CR_ADDR[1:0]	H_AOUT[3:2]
CR_ADS#	H_ADSM#
CR_BE#[3:0]	H_BEOUT#[3:0]
CR_DRD[31:0]	H_DIN[3:0]
CR_DWR[31:0]	H_DOUT[3:0]
CR_RDY#	H_RDYH#
CR_WR	H_WROUT
"1"	H_ABORT#

**Table 21: DMA Programming Port**

DMA programing port	Target side user interface
“0”	H_BII#
Open	H_BLAST#
Open	H_BOFF#
Open	H_CS#
“1”	H_RETRY#
D_INTR#	H_INTA# *

* D_INTR# and H_INTA# are optional connection. This would trigger interrupt on the PCI bus when DMA completes.

Eureka Technology Inc.
4962 El Camino Real
Los Altos, CA 94022, USA

Tel: 1 650 960 3800
Fax: 1 650 960 3805
email: info@eurekatech.com
<http://www.eurekatech.com>